



Helwan University

Faculty of Science

Department of Computer Science and Statistics

Faculty

Logo

Project

Logo

FINAL YEAR GRADUATION PROJECT

NABTA

Plant Disease Analysis System

An End-to-End Artificial Intelligence Platform for Plant Health Analysis

TEAM MEMBERS

Ziad Walid Mokhtar	—
Sherif Karam Zaki	352250808
Armia Gamal Fakry	—
Peter Bolbol	—
Salsabel Esmail Hussin	—
Sara Essam Ibrahim	—
Shada Ayman Eldin Ahmed	—

SUPERVISED BY

Dr. Mourad Raafat

*To the farmers of the world,
whose tireless labour feeds nations
and whose challenges inspired this work.*

*And to our families,
whose unwavering belief in us
made every obstacle surmountable.*

Acknowledgements

The authors gratefully acknowledge the guidance and unwavering support of **Dr. Mourad Raafat**, whose expert supervision, constructive criticism, and continuous encouragement were instrumental throughout every phase of this project's development. His deep knowledge of artificial intelligence and software engineering fundamentally shaped the quality of this work.

The team extends sincere appreciation to the **Department of Computer Science and Statistics** at the **Faculty of Science, Helwan University**, for providing an intellectually stimulating academic environment and the computational resources necessary to conduct the extensive model training experiments described herein.

Gratitude is likewise extended to the open-source communities and academic researchers behind the datasets, frameworks, and tools that form the technical foundation of NABTA, including the contributors to PlantVillage, PlantDoc, the Ultralytics YOLOv8 ecosystem, and the underlying deep learning frameworks employed during model development. Without their freely shared contributions, projects of this nature would not be possible.

Finally, the authors dedicate sincere thanks to their families for their patience, moral support, and encouragement throughout the duration of this project.

Abstract

This project introduces NABTA, an intelligent, full-stack, production-ready, end-to-end artificial intelligence system designed for automated plant care and pathogen diagnosis. The proposed platform addresses a critical and globally pervasive challenge in modern agriculture: the timely and accurate identification of plant diseases under practical field conditions, a challenge responsible for the destruction of up to **40%** of global crop yields on an annual basis.

The NABTA architecture integrates multiple computer vision models in a sequential pipeline. You Only Look Once version 8 (YOLOv8) performs real-time object detection to isolate leaf regions using high-confidence bounding boxes. A custom Convolutional Neural Network (CNN) subsequently conducts fine-grained classification across **102** disease and healthy plant classes. Pixel-level segmentation is achieved using a U-Net architecture with an EfficientNet-B0 encoder, producing refined tissue masks with a mean Intersection over Union (IoU) of **91.8%**. Removing background artefacts – including soil, shadows, and unrelated vegetation – before classification is a central contribution of the multi-stage pipeline design.

To address the interpretability gap prevalent in black-box deep learning systems, the platform incorporates Gradient-weighted Class Activation Mapping (Grad-CAM) to generate gradient-weighted heatmaps that visually justify the model's classification decisions to non-technical end-users. Furthermore, a domain-restricted generative Artificial Intelligence (AI) chatbot, powered by the Cohere Application Programming Interface (API) and Retrieval-Augmented Generation (RAG), provides expert-quality treatment prescriptions and agronomist-vetted consultations in both Arabic and English. All inference services are exposed through a high-performance, asynchronous FastAPI backend backed by a vector database, while the user-facing layer is delivered as a fully responsive React.js dashboard featuring historical analysis records, infection timeline visualisations, and a tiered subscription model.

Empirical evaluation demonstrates a CNN classification accuracy of **96.4%**, a YOLOv8 mean Average Precision (mean Average Precision (mAP)) of **94.1%**, and a U-Net IoU of **91.8%**, with end-to-end inference completing in under **0.4 seconds**. The Large Language Model (LLM) consultation module achieved a satisfaction score of **4.7 out of 5.0** as assessed by independent agricultural experts under blind evaluation conditions. These results collectively affirm the viability of NABTA as a deployable, scalable precision-agriculture solution capable of mitigating crop losses attributable to delayed or inaccurate disease diagnosis.

Keywords: Plant disease detection, deep learning, YOLOv8, U-Net, convolutional neural networks, Grad-CAM, explainable AI, retrieval-augmented generation, FastAPI, smart agriculture, precision farming.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
List of Abbreviations	xvi
1 Introduction	1
1.1 Background and Context.....	1
1.2 Problem Statement.....	2
1.3 Motivation	3
1.4 Project Objectives	4
1.5 Project Scope.....	5
1.6 Contributions.....	5
1.7 Document Organisation.....	6
2 Literature Review and System Analysis	7
2.1 Related Work.....	7
2.2 Overview	7
2.3 Stakeholder Analysis	8
2.4 User Roles	8

2.4.1	Free-Tier User	9
2.4.2	Pro-Tier User	9
2.4.3	System Administrator	9
2.5	Functional Requirements	9
2.5.1	Authentication and Account Management	10
2.5.2	Plant Image Analysis	11
2.5.3	AI Consultation Chatbot	12
2.5.4	Analytics Dashboard	13
2.6	Non-Functional Requirements	14
2.7	Use Cases	15
2.7.1	UC-01: User Registration	15
2.7.2	UC-02: Plant Image Upload and Analysis	16
2.7.3	UC-03: AI Chatbot Consultation	17
2.7.4	UC-04: PDF Report Generation	18
2.7.5	UC-05: Subscription Upgrade	18
2.8	Test Cases	19
3	Methodology	21
3.1	Overview	21
3.2	Data Collection	21
3.2.1	Dataset Strategy	21
3.2.2	Component Datasets	22
3.2.3	Detection-Specific Corpus: PlantDoc Enhancement	23
3.2.4	Segmentation Mask Construction	23
3.3	Data Preprocessing	23
3.3.1	Image Resizing and Normalisation	23
3.3.2	Data Integrity Filtering	24

3.3.3	Data Augmentation	24
3.4	Feature Engineering.....	25
3.5	Model Development.....	26
3.5.1	YOLOv8 Object Detection Model	26
3.5.2	U-Net Semantic Segmentation Model	27
3.5.3	Custom CNN Classification Model	27
3.6	Grad-CAM Explainability Module	29
3.7	RAG-Augmented Consultation Pipeline.....	30
3.8	Validation Strategy	30
4	System Design.....	31
4.1	Overview	31
4.2	System Architecture	31
4.3	Design Artefact Placeholders	32
4.4	Frontend Design	34
4.4.1	Technology Stack	34
4.4.2	Key Interface Screens.....	34
4.4.2.1	Main Analysis Dashboard	34
4.4.2.2	Smart Analysis Center	35
4.4.2.3	AI Chat Assistant	36
4.5	Backend Design	37
4.5.1	Technology Stack	37
4.5.2	API Router Structure	37
4.5.3	Image Preprocessing Module	37
4.5.4	ML Pipeline Router (MLManager)	38
4.6	Database Design	38

4.6.1	Firestore Document Schema	38
4.6.2	Vector Database Schema	39
4.7	API Specification	40
4.8	Security Architecture	40
4.8.1	Authentication and Authorisation	40
4.8.2	Transport Security	41
4.8.3	API Key Management	41
4.8.4	Input Validation	41
5	Implementation	42
5.1	Overview	42
5.2	Technology Stack	42
5.2.1	AI and Machine Learning Technologies	43
5.2.2	Backend Technologies	44
5.2.3	Frontend Technologies	45
5.2.4	Infrastructure and DevOps	46
5.3	Development Environment	46
5.4	Key Implementation Details	46
5.4.1	Model Serialisation and Loading	47
5.4.2	Asynchronous Pipeline Execution	47
5.4.3	Grad-CAM Implementation	48
5.4.4	RAG Knowledge Base Ingestion	48
5.4.5	Internationalisation (i18n)	49
5.5	Deployment	49
5.5.1	Backend Deployment	49
5.5.2	Frontend Deployment	49
5.5.3	CI/CD Pipeline	50

5.6	System Screenshots	50
6	Results and Evaluation	52
6.1	Overview	52
6.2	Classification Model Performance	52
6.2.1	Overall Accuracy	52
6.2.2	Per-Class Performance Highlights	53
6.2.3	Training Convergence.	54
6.3	Object Detection Model Performance	55
6.4	Segmentation Model Performance	56
6.5	End-to-End Pipeline Performance	57
6.6	LLM Consultation Evaluation	57
6.7	Comparative Analysis	58
6.8	Testing Summary	59
6.9	Discussion	60
7	Conclusion and Future Work	61
7.1	Summary of Achievements	61
7.2	Known Limitations	62
7.3	Future Work	63
7.3.1	Native Mobile Application	63
7.3.2	Edge AI Deployment via TensorFlow Lite	63
7.3.3	Multimodal Input Synthesis	64
7.3.4	Continuous Disease Severity Quantification.	64
7.3.5	Autonomous Farm Management Integration	64
7.3.6	Federated Learning for Continuous Improvement	64
7.3.7	Active Learning for Out-of-Distribution Detection	65

7.4	Closing Remarks	65
A	Dataset Details and Class Taxonomy	66
A.1	Complete List of 102 Target Classes	66
A.2	Dataset Class Distribution	71
B	Project Visual Evidence Placeholders	72
	Bibliography	74
	Glossary	76

List of Figures

4.1	High-Level Three-Tier System Architecture of NABTA	32
4.2	System Workflow Diagram	33
4.3	Model and Inference Pipeline	33
4.4	Database Schema Diagram	34
4.5	NABTA Main Analysis Dashboard Interface	35
4.6	NABTA Smart Analysis Center Dashboard	36
4.7	NABTA AI Chat Assistant Interface	36
5.1	Image Upload Screen with File Type and Size Indicators	50
5.2	Analysis Results Screen with Detection, Segmentation, and Grad-CAM Panels .	51
5.3	Subscription Tier Selection Screen	51
6.1	CNN Training and Validation Accuracy and Loss over 100 Epochs	55
6.2	YOLOv8 Precision-Recall Curve on the PlantDoc Test Set	56
6.3	Evaluation Results Summary	59
6.4	Model Performance Comparison	60
A.1	Distribution of Images Across Crop Categories in the Combined Training Corpus	71
B.1	Database Schema Diagram	72
B.2	User Interface Screenshot Set	73

B.3 Model Performance Comparison	73
--	----

List of Tables

2.1	Stakeholder Groups and Their Primary Interests	8
2.2	Functional Requirements: Authentication and Account Management	10
2.3	Functional Requirements: Plant Image Analysis	11
2.4	Functional Requirements: AI Consultation Chatbot	12
2.5	Functional Requirements: Analytics Dashboard	13
2.6	Non-Functional Requirements	14
2.7	Use Case: User Registration	15
2.8	Use Case: Plant Image Upload and Analysis	16
2.9	Use Case: AI Chatbot Consultation	17
2.10	Use Case: PDF Report Generation	18
2.11	Use Case: Subscription Upgrade	18
2.12	System Test Case Inventory (Part 1 of 2)	19
2.13	System Test Case Inventory (Part 2 of 2)	20
3.1	Training Dataset Components for the NABTA Classification Model	22
3.2	Data Augmentation Parameters Applied During Training	25
3.3	YOLOv8 Training Hyperparameters	26
3.4	U-Net Training Hyperparameters	27
3.5	Custom CNN Classification Model Training Hyperparameters	29

4.1	FastAPI Router Structure	37
4.2	Firestore Collection Schema	39
4.3	NABTA REST API Endpoint Reference	40
5.1	AI and Machine Learning Technologies Employed	43
5.2	Backend Technologies Employed	44
5.3	Frontend Technologies Employed	45
5.4	Infrastructure and DevOps Technologies	46
6.1	Custom CNN Classification Model — Aggregate Test Set Metrics	53
6.2	CNN Per-Class Performance (Selected Classes)	54
6.3	YOLOv8 Detection Model — Test Set Metrics	55
6.4	U-Net Segmentation Model — Test Set Metrics	56
6.5	End-to-End Pipeline Latency Breakdown	57
6.6	Expert Panel LLM Consultation Evaluation Scores (Likert 1–5)	58
6.7	Comparative Performance — Plant Disease Classification Systems	58
6.8	Testing Summary	59
A.1	Complete 102-Class Label Taxonomy (Part 1 of 4)	67
A.2	Complete 102-Class Label Taxonomy (Part 2 of 4)	68
A.3	Complete 102-Class Label Taxonomy (Part 3 of 4)	69
A.4	Complete 102-Class Label Taxonomy (Part 4 of 4)	70

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

ASGI Asynchronous Server Gateway Interface

BCE Binary Cross-Entropy

CD Continuous Deployment

CDN Content Delivery Network

CI Continuous Integration

CNN Convolutional Neural Network

CPU Central Processing Unit

DL Deep Learning

ERD Entity-Relationship Diagram

FAO Food and Agriculture Organization of the United Nations

GAP Global Average Pooling

GPU Graphics Processing Unit

Grad-CAM Gradient-weighted Class Activation Mapping

HSTS HTTP Strict Transport Security

IoU Intersection over Union

JWT	JSON Web Token
LLM	Large Language Model
mAP	mean Average Precision
MIME	Multipurpose Internet Mail Extensions
ML	Machine Learning
NLP	Natural Language Processing
NMS	Non-Maximum Suppression
ORM	Object-Relational Mapping
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
REST	Representational State Transfer
ROI	Region of Interest
SDK	Software Development Kit
SPA	Single-Page Application
SUS	System Usability Scale
TFLite	TensorFlow Lite
U-Net	U-shaped Encoder-Decoder Network
UAT	User Acceptance Testing
UML	Unified Modeling Language
WSGI	Web Server Gateway Interface
XAI	Explainable Artificial Intelligence
YOLOv8	You Only Look Once version 8

Introduction

1.1 Background and Context

Agriculture constitutes one of the most fundamental pillars of human civilisation, providing sustenance, economic activity, and employment to billions of people across all continents. Despite dramatic advances in industrial food production and globalised supply chains, the security of the world's crop yields remains perpetually vulnerable to biological threats. Among these threats, plant pathogens—encompassing fungal infections, bacterial blights, viral diseases, and parasitic infestations—represent perhaps the most persistent and damaging challenge facing agricultural communities worldwide.

The Food and Agriculture Organization of the United Nations (FAO) estimates that plant diseases are collectively responsible for the destruction of up to **40%** of all food crops globally on an annual basis [1]. This staggering figure translates not merely to economic losses measured in billions of dollars, but to food insecurity for vulnerable populations, reduced livelihoods for smallholder farmers, and the unnecessary over-application of chemical pesticides that degrades soil health and contaminates water systems. The problem is particularly acute in developing nations, where limited access to agronomic expertise and diagnostic infrastructure renders early intervention exceptionally difficult.

Historically, the identification and management of plant diseases has depended heavily on the manual inspection capabilities of trained agronomists and plant pathologists. Such experts evaluate macroscopic and microscopic symptoms—discolouration patterns, lesion morphology, necrotic tissue boundaries, and spore structures—to arrive at a differential diagnosis. This approach, while clinically valid, suffers from three fundamental operational limitations: it is

time-intensive, requiring sustained expert availability; it is *geographically constrained*, given the scarcity of credentialed specialists in many agricultural regions; and it is inherently *non-scalable*, since even an experienced agronomist can examine only a limited number of field samples within a practical timeframe.

The convergence of deep learning, computer vision, and cloud computing over the past decade has opened genuinely transformative possibilities for automated, scalable, and accessible plant disease diagnosis. Landmark work such as that of Mohanty et al. [2] demonstrated that convolutional neural networks trained on curated leaf-image datasets could achieve accuracy levels competitive with—and in some cases exceeding—those of human experts under controlled photographic conditions. Subsequent advances in object detection architectures, notably the YOLO family [3], and in semantic segmentation networks, most prominently the U-Net architecture [4], have further expanded the toolkit available for intelligent agricultural analysis. Alongside these discriminative models, the emergence of large language models and retrieval-augmented generation [5] has created new possibilities for deploying expert knowledge at scale through conversational interfaces.

1.2 Problem Statement

Despite the technological advances described above, a substantial gap persists between isolated academic demonstrations and practical, deployed systems that farmers and agricultural workers can readily access and trust. Several specific, interconnected challenges characterise this gap and motivated the development of the NABTA system:

- P1. Stage-Specific Vulnerability:** Plant pathogens typically exhibit their most characteristic visual symptoms only in intermediate or advanced disease stages. Early-stage infections, when intervention would be most cost-effective, present subtle lesion patterns that are easily overlooked by non-specialist observers or overwhelmed by the complex visual noise present in field photographs—soil backgrounds, uneven lighting, overlapping foliage, and shadow gradients.
- P2. Expert Scarcity and Geographic Inaccessibility:** Qualified plant pathologists and agronomists are unevenly distributed across agricultural geographies. Rural and remote farming communities in developing nations face the most acute shortfall, with access to

specialist consultations requiring either costly field visits or lengthy delays.

- P3. Financial Constraints of Manual Diagnosis:** Continuous reliance on human expert evaluation is economically unsustainable for smallholder farmers operating on tight margins. Laboratory analysis, travel costs, and specialist fees collectively impose a recurring financial burden that suppresses diagnostic uptake.
- P4. Epidemic Spreading and Delayed Response:** Without rapid, on-demand diagnostic capabilities, infected plants may go undetected for extended periods. During this interval, pathogen spores and vectors spread to neighbouring plants and adjacent fields, escalating localised infections into wide-scale epidemic events before any containment response is mounted.
- P5. Misdiagnosis Risk and Pesticide Misuse:** Incorrect identification of a pathogen, whether by an under-qualified observer or by a single-stage, context-insensitive AI model, leads directly to the application of inappropriate chemical treatments. Such misuse wastes agricultural inputs, accelerates the development of chemical resistance among pathogen populations, and introduces unnecessary environmental contamination.
- P6. Scalability Ceiling of Human-Dependent Systems:** Traditional diagnostic workflows cannot be linearly scaled to monitor extensive agricultural operations. Large farms with hundreds of hectares under cultivation require a form of continuous, automated surveillance that no human workforce can economically provide.

1.3 Motivation

The motivation for developing NABTA arises from the recognition that the individual technologies required to address the above problems already exist in mature or near-mature forms within the research literature. What is absent is a coherent, production-quality system that combines these technologies in a principled architectural framework, exposes them through an accessible user interface suitable for non-technical agricultural workers, and deploys them with the security, reliability, and performance characteristics demanded by real-world operational environments.

Furthermore, the authors were motivated by the observation that existing plant disease classification systems overwhelmingly adopt single-stage pipelines—one model receives a raw field

image and produces a class label. This design choice, while computationally economical, fundamentally ignores the compound degradation in model confidence caused by complex photographic backgrounds. A hierarchical pipeline that progressively refines the input—first isolating the leaf region, then segmenting its tissue boundaries, and finally classifying the isolated pathological features—constitutes a significantly more robust approach to real-world conditions.

1.4 Project Objectives

The NABTA project pursues the following primary engineering and research objectives:

- O1. Unified Multi-Model AI Pipeline:** Design and implement a sequential, multi-stage computer vision pipeline that combines object detection (YOLOv8), semantic segmentation (U-Net), and image classification (CNN) into a cohesive inference workflow acting on a single raw input image.
- O2. High-Accuracy Pathogen Classification:** Achieve classification accuracy exceeding 95% across a dataset spanning 102 distinct disease categories and healthy plant states, encompassing a diverse range of crop species and pathogen types.
- O3. Explainable AI Integration:** Incorporate Grad-CAM visualisations into the inference output to ensure that model decisions are interpretable and verifiable by agronomists and non-technical users alike, thereby building operational trust in the system.
- O4. Intelligent Consultation Layer:** Develop a domain-restricted, multilingual conversational AI module using RAG and the Cohere LLM platform, capable of delivering expert-quality treatment prescriptions grounded in curated agricultural knowledge.
- O5. Production-Ready Cloud Deployment:** Architect a high-performance asynchronous backend using FastAPI, capable of servicing concurrent field requests with sub-second inference latency.
- O6. Accessible, Responsive Interface:** Deliver a fully featured, mobile-responsive web application in React.js that enables farmers with minimal technical literacy to submit plant images and receive comprehensive diagnostic reports.

O7. Robust Security Architecture: Implement a multi-layered security posture encompassing Firebase Authentication, HTTPS encryption, and secure API key management.

1.5 Project Scope

The scope of the NABTA project encompasses the full software development lifecycle of the described system, from requirements analysis and architectural design through model training, system integration, and deployment. The project operates within the following defined boundaries:

The classification capability is bounded by the training data: the system diagnoses plant diseases within the 102 classes represented in the combined training corpus, which includes crops such as tomato, potato, pepper, apple, grape, corn, rice, cotton, mango, eggplant, strawberry, orange, and cucumber, among others. Extension to additional species and pathogen types is identified as a future research direction.

The system is designed for *leaf-level image analysis*. Diagnosis derived from full-plant or field-view imagery without sufficient leaf prominence is not the primary design target, although the YOLOv8 detection stage partially compensates for this by isolating leaf-level regions of interest even within complex scene compositions.

The chatbot consultation interface is intentionally domain-restricted. The system rejects queries outside the agricultural domain to maintain response reliability and prevent the dissemination of hallucinated advice for off-topic subjects.

1.6 Contributions

The principal technical contributions of the NABTA project are summarised as follows:

- **End-to-End Multi-Stage Pipeline:** A novel sequential pipeline architecture combining YOLOv8 detection, U-Net segmentation, and CNN classification, demonstrated to achieve superior robustness over single-stage baselines on field-condition imagery.
- **Large-Scale Training Corpus:** The curation and integration of a combined training dataset exceeding 120,000 images across 102 classes, sourced from seven distinct open-source agri-

cultural repositories, with systematic balancing and augmentation.

- **Integrated Grad-CAM Explainability:** The first demonstration, to the authors' knowledge, of real-time Grad-CAM overlay delivery within a production web application targeting agricultural end-users.
- **RAG-Augmented Agricultural Chatbot:** A deployable, domain-restricted conversational AI module providing treatment recommendations grounded in a curated vector knowledge base, evaluated and validated by agricultural domain experts.
- **Production Deployment Architecture:** A complete, documented system architecture combining FastAPI, Firebase, a vector database, and React.js, designed for immediate deployability in cloud environments.

1.7 Document Organisation

The remainder of this document is structured as follows. Chapter 2 presents the domain background, related work, requirements analysis, and user-facing use cases. Chapter 3 details the data collection strategy, preprocessing protocols, training corpus composition, and model development methodology. Chapter 4 describes the full system architecture, backend design, frontend design, database schema, API specification, and modelling artefact placeholders. Chapter 5 documents the technologies employed, the development environment, and key implementation details with illustrative screenshot placeholders. Chapter 6 presents evaluation metrics, testing evidence, and comparative discussion. Finally, Chapter 7 synthesises the project's achievements, acknowledges its limitations, and outlines the roadmap for future enhancement.

Literature Review and System Analysis

2.1 Related Work

Plant disease diagnosis has evolved from manual visual inspection toward computer vision systems that can classify disease symptoms from leaf images. Early deep learning studies, including Mohanty et al. [2], demonstrated the feasibility of convolutional neural networks on controlled datasets such as PlantVillage. Later work introduced stronger backbones, object detection models, segmentation networks, and explainability techniques to improve robustness under field conditions.

The NABTA project builds on this body of work by combining object detection, segmentation, classification, and retrieval-augmented consultation in one deployable workflow. Unlike single-stage classifiers that process the whole image directly, the proposed pipeline progressively isolates the leaf region, removes background noise, classifies the disease, and explains the decision through Grad-CAM.

2.2 Overview

System analysis constitutes the foundational phase in which the problem domain is formally examined and translated into a structured set of requirements that guide all subsequent design and implementation activities. This chapter presents a comprehensive requirements analysis for the NABTA system, encompassing both functional and non-functional specifications, a characterisation of the system's stakeholder groups and user roles, and a structured inventory of use cases and test scenarios.

The analysis was conducted through a combination of literature review, user-centric need identification, and iterative refinement by the development team in consultation with the project supervisor. The resulting requirements document served as the primary contract specification governing system development.

2.3 Stakeholder Analysis

The NABTA system serves a heterogeneous community of stakeholders with distinct interests and interaction modalities:

Table 2.1. Stakeholder Groups and Their Primary Interests

Stakeholder	Role	Primary Interest
Field Farmers	Primary end-users	Rapid, reliable disease identification for crop protection
Agronomists	Domain experts and validators	Verifiable, evidence-based diagnostic output
Agricultural Researchers	Secondary users	Access to aggregated disease distribution data
System Administrators	Technical operators	System uptime, security, and performance monitoring
Project Supervisors	Academic oversight	Research quality and documentation standards
Helwan University	Institutional sponsor	Demonstrable academic and practical contribution

2.4 User Roles

The system defines three distinct user roles with differentiated access privileges:

2.4.1 Free-Tier User

The free-tier user represents the baseline access level, available to all registered accounts at no cost. Free-tier users may upload plant images for sequential pipeline analysis, view the resulting detection bounding boxes, segmentation masks, classification labels, confidence scores, and Grad-CAM heatmaps. Access to the AI chatbot consultation interface is provided with daily usage limitations. Historical analysis records are retained for a defined rolling window period.

2.4.2 Pro-Tier User

Pro-tier users (\$20/month) access the full feature set without usage restrictions. Additional capabilities exclusive to pro-tier accounts include unrestricted daily analysis submissions, the ability to upload custom datasets for personalised model fine-tuning, access to the comprehensive Nabta Smart Analysis Center dashboard with detailed farm-level infection timeline visualisations, and the export of analytical reports in PDF format.

2.4.3 System Administrator

System administrators possess elevated privileges for user management, system health monitoring, model version management, and content moderation within the knowledge base used by the RAG consultation module.

2.5 Functional Requirements

Functional requirements specify the behavioural capabilities that the NABTA system must provide. Each requirement is assigned a unique identifier for traceability.

2.5.1 Authentication and Account Management

Table 2.2. Functional Requirements: Authentication and Account Management

ID	Priority	Requirement Description
FR-01	High	The system shall allow new users to register accounts via an email and password mechanism facilitated by Firebase Authentication.
FR-02	High	The system shall authenticate registered users via secure login and maintain authenticated session state.
FR-03	High	The system shall provide a password reset workflow via a registered email address.
FR-04	Medium	The system shall send a welcome email to newly registered users upon successful account creation.
FR-05	Medium	The system shall maintain differentiated access controls between free-tier and pro-tier user accounts.

2.5.2 Plant Image Analysis

Table 2.3. Functional Requirements: Plant Image Analysis

ID	Priority	Requirement Description
FR-06	High	The system shall accept plant leaf images in JPEG and PNG formats up to a maximum file size of 5 MB.
FR-07	High	The system shall perform object detection on uploaded images using the YOLOv8 model to identify and bound infected leaf regions.
FR-08	High	The system shall perform semantic segmentation on extracted ROI crops using the U-Net model with EfficientNet-B0 encoder.
FR-09	High	The system shall classify the segmented and refined leaf image into one of 102 target disease/healthy classes using the custom CNN.
FR-10	High	The system shall generate Grad-CAM gradient heatmaps overlaid on the classification input image.
FR-11	High	The system shall return the plant name, disease name, confidence score, infection severity percentage, and Grad-CAM overlay image to the client.
FR-12	High	The system shall store all analysis results associated with the authenticated user's account.
FR-13	Medium	The system shall generate and allow download of a PDF report summarizing the diagnosis.

2.5.3 AI Consultation Chatbot

Table 2.4. Functional Requirements: AI Consultation Chatbot

ID	Priority	Requirement Description
FR-14	High	The system shall provide a conversational AI interface for agricultural queries powered by the Cohere LLM and RAG pipeline.
FR-15	High	The system shall restrict chatbot responses to agricultural domain queries, rejecting off-topic prompts with a polite refusal message.
FR-16	High	The system shall support both Arabic and English input and response languages.
FR-17	High	The chatbot shall provide structured responses covering overview, cause, infection analysis, severity status, organic treatment, chemical treatment, prevention guidelines, and warning signs.
FR-18	Medium	The chatbot shall generate a final health score and priority action list at the conclusion of each consultation session.

2.5.4 Analytics Dashboard

Table 2.5. Functional Requirements: Analytics Dashboard

ID	Priority	Requirement Description
FR-19	High	The system shall display a disease distribution chart summarising historical analysis counts by disease type.
FR-20	High	The system shall display an infection timeline chart enabling users to observe seasonal and temporal infection trends.
FR-21	Medium	The system shall display a ranking of the most frequently analysed plant species.
FR-22	Medium	The system shall maintain and display a paginated log of recent analyses with timestamps, species, and disease tags.
FR-23	Low	Pro-tier users shall be able to export dashboard data in standard tabular formats.

2.6 Non-Functional Requirements

Table 2.6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The end-to-end inference pipeline shall complete within 0.4 seconds for standard resolution inputs under nominal server load.
NFR-02	Scalability	The backend shall support concurrent multi-user inference requests through asynchronous processing.
NFR-03	Accuracy	The CNN classifier shall achieve a minimum accuracy of 95% on the held-out validation set at release.
NFR-04	Reliability	The system shall maintain an uptime target of at least 99.5% during active deployment.
NFR-05	Security	All client-server communication shall be encrypted via HTTPS. User authentication shall be managed through Firebase, and API keys shall be stored securely using environment variable vaults.
NFR-06	Usability	The web interface shall be fully functional on modern desktop and mobile browsers with no additional plugin installation required.
NFR-07	Maintainability	System components shall be modularly decoupled such that individual model nodes can be upgraded independently without requiring refactoring of frontend or unrelated backend modules.
NFR-08	Accessibility	The interface shall support bilingual (Arabic and English) content rendering.
NFR-09	Portability	The backend shall be containerisable via Docker for cloud-agnostic deployment.

2.7 Use Cases

2.7.1 UC-01: User Registration

Table 2.7. Use Case: User Registration

Field	Details
Use Case ID	UC-01
Name	User Registration
Actor(s)	New User
Preconditions	User has not previously registered an account
Main Success Scenario	User submits registration form → Firebase Auth validates credentials → Account created → Welcome email dispatched
Alternative Scenario	Email already in use → System displays error; user prompted to log in or reset password
Postconditions	Authenticated user session established; user redirected to dashboard

2.7.2 UC-02: Plant Image Upload and Analysis

Table 2.8. Use Case: Plant Image Upload and Analysis

Field	Details
Use Case ID	UC-02
Name	Plant Image Upload and Analysis
Actor(s)	Authenticated User (Free or Pro Tier)
Preconditions	User is logged in; image file meets format and size requirements
Main Success Scenario	User uploads image → Backend preprocesses image → YOLOv8 detects ROI → U-Net segments leaf → CNN classifies disease → Grad-CAM generated → Results returned to frontend → Results stored in database
Alternative Scenario A	No leaf detected by YOLO → System returns informative error advising improved image quality
Alternative Scenario B	File exceeds 5MB limit → Upload rejected with size-limit notification
Postconditions	Analysis result persisted; user viewing diagnosis report

2.7.3 UC-03: AI Chatbot Consultation

Table 2.9. Use Case: AI Chatbot Consultation

Field	Details
Use Case ID	UC-03
Name	AI Chatbot Consultation
Actor(s)	Authenticated User
Preconditions	User is logged in; daily query limit not exceeded
Main Success Scenario	User submits agricultural query → Query processed by RAG pipeline → Relevant documents retrieved from vector DB → Cohere LLM generates structured response → Response rendered in chat panel
Alternative Scenario	Non-agricultural query submitted → Domain restriction filter activates; polite refusal returned
Postconditions	Response displayed; conversation history maintained in session

2.7.4 UC-04: PDF Report Generation

Table 2.10. Use Case: PDF Report Generation

Field	Details
Use Case ID	UC-04
Name	PDF Report Generation
Actor(s)	Authenticated User
Preconditions	At least one analysis result exists for the user
Main Success Scenario	User requests PDF report → Backend compiles diagnosis, images, and treatment information → PDF generated and returned as download
Postconditions	PDF file downloaded to user's device

2.7.5 UC-05: Subscription Upgrade

Table 2.11. Use Case: Subscription Upgrade

Field	Details
Use Case ID	UC-05
Name	Pro-Tier Subscription Upgrade
Actor(s)	Free-Tier User
Preconditions	User is logged in with a free-tier account
Main Success Scenario	User navigates to upgrade page → Payment processed → Account tier updated in Firebase → Pro features unlocked
Extended Feature	Unlocks custom dataset upload for personalised model training
Postconditions	User account upgraded; pro-tier dashboard features accessible

2.8 Test Cases

Table 2.12. System Test Case Inventory (Part 1 of 2)

TC ID	Linked UC	Test Description	Expected Outcome
TC-01	UC-01	Submit registration with a valid email and strong password	Account created; welcome email received
TC-02	UC-01	Submit registration with an existing email	Error displayed; no duplicate account created
TC-03	UC-02	Upload a clear tomato leaf image with visible blight lesions	Early blight classified with $\geq 90\%$ confidence
TC-04	UC-02	Upload an image exceeding 5 MB	Upload rejected with an appropriate message
TC-05	UC-02	Upload an image without visible leaf content	No ROI returned; user prompted to retry
TC-06	UC-02	Upload a healthy leaf image	Healthy class label returned
TC-07	UC-02	Verify that the Grad-CAM overlay accompanies the result	Heatmap returned and rendered by the frontend

Table 2.13. System Test Case Inventory (Part 2 of 2)

TC ID	Linked UC	Test Description	Expected Outcome
TC-08	UC-03	Ask about treatment for a tomato fungal disease	Organic and chemical treatment sections returned
TC-09	UC-03	Submit a non-agricultural query	Polite domain-restriction response returned
TC-10	UC-03	Submit a query in Arabic	Response returned in Arabic
TC-11	UC-04	Request a PDF report after an analysis	Report downloaded with the diagnosis summary
TC-12	UC-05	Upgrade a free account through the payment flow	Pro features enabled; usage limits updated
TC-13	NFR-01	Measure total pipeline latency for a standard image	Response time ≤ 400 ms
TC-14	NFR-05	Inspect network traffic during authentication	HTTPS used; no plaintext credentials observed

Methodology

3.1 Overview

The methodological framework underlying the NABTA system reflects a principled, iterative approach to the development and validation of machine learning models for agricultural pathogen diagnosis. This chapter details each stage of the data science lifecycle as applied to the project: dataset collection and curation, preprocessing and augmentation strategies, the architecture and training of each component model, and the validation strategy employed to evaluate generalisation performance. The design decisions described herein were guided by a core objective of maximising real-world robustness rather than benchmark-optimised performance on curated laboratory data.

3.2 Data Collection

3.2.1 Dataset Strategy

The fundamental challenge in building a robust plant disease classifier is not merely the acquisition of data, but the acquisition of sufficiently diverse data that represents the heterogeneity of real field conditions. A single-source dataset, however large, tends to encode the photographic style, lighting conditions, and crop species distribution of its contributing institution. To mitigate this domain-bias problem, the NABTA training corpus was assembled by integrating seven distinct publicly available agricultural image repositories.

The combined dataset totals in excess of **120,000 annotated images** spanning **102 target classes**—comprising disease states across multiple crop species as well as corresponding

healthy class labels for each species. This multi-source fusion constitutes a substantially more challenging and representative training distribution than any single component dataset.

3.2.2 Component Datasets

Table 3.1. Training Dataset Components for the NABTA Classification Model

Dataset	Description	Focus	Source
PlantVillage	Benchmark dataset of 54,305 images across 14 crops and 26 diseases; laboratory-controlled conditions	Classification	Kaggle / Penn State
PlantDoc	2,569 images collected under diverse outdoor conditions; includes 13 plant species and 17 disease classes	Detection & Classification	Open Source
Plant Diseases Training Dataset	Supplementary multi-crop collection for generalisation improvement	Classification	Kaggle
Bangladesh Crop Dataset	Region-specific rice and jute crop images for geographic diversity	Classification	Kaggle
Rice Leaf Disease Dataset	Dedicated collection for bacterial blight, blast, and tungro diseases in rice	Classification	Kaggle
Cotton Dataset	Images of cotton leaf curl virus, bacterial blight, and healthy states	Classification	Kaggle
Mango Plant Dataset	Anthraxnose, powdery mildew, and healthy mango leaf images	Classification	Kaggle
Eggplant Pathogen Dataset	Eggplant-specific disease images including phomopsis blight	Classification	Kaggle
Custom Segmentation Masks	Pixel-level binary masks constructed on top of classified leaf images	Segmentation	In-house annotation

3.2.3 Detection-Specific Corpus: PlantDoc Enhancement

The PlantDoc dataset [6] was selected as the primary training source for the YOLOv8 object detection model due to its outdoor, unconstrained photographic conditions. Raw PlantDoc annotations were manually reviewed and significantly enhanced: bounding box coordinates were refined using a combination of semi-automated annotation tools and manual correction to ensure tight, accurate ROI boundaries. Images with excessive motion blur, severe occlusion, or ambiguous ground truth labels were removed from the training split to prevent noise injection into the detection model's weight optimisation.

3.2.4 Segmentation Mask Construction

No off-the-shelf pixel-level segmentation dataset aligned precisely with the species distribution required by NABTA. Consequently, a custom set of binary segmentation masks was constructed by the project team. The mask creation workflow proceeded as follows: a subset of classified leaf images was selected from the combined corpus; each image was processed through a combination of GrabCut background subtraction and manual polygon annotation to produce a clean binary mask isolating the leaf body from background content. The resulting mask set was used exclusively for U-Net training, with the leaf foreground designated as the positive class and all background pixels designated as the negative class.

3.3 Data Preprocessing

3.3.1 Image Resizing and Normalisation

All images in the classification training corpus were resized to a uniform spatial resolution of 224×224 pixels to match the input dimensionality requirements of the custom CNN architecture. Images for the U-Net segmentation model were resized to 256×256 pixels. YOLOv8 accepts variable resolution inputs; training images were resized to 640×640 pixels with letterboxing to preserve aspect ratios.

Pixel intensity values were normalised to the $[0, 1]$ range via division by 255. Channel-wise mean subtraction and standard deviation scaling were applied using the ImageNet mean ($\mu = [0.485, 0.456, 0.406]$) and standard deviation ($\sigma = [0.229, 0.224, 0.225]$) for transfer-learning

compatibility.

3.3.2 Data Integrity Filtering

Before training, the full combined corpus was subjected to an automated data integrity scan. Images were rejected if they met any of the following disqualification criteria:

- File corruption or incomplete read (zero-byte or truncated files).
- Laplacian variance below a blur threshold of 50.0, indicating excessive motion blur or defocus.
- Aspect ratio exceeding 5:1 in either dimension, suggesting severely non-standard capture conditions.
- Class label mismatch identified during cross-dataset harmonisation.

Approximately 3.2% of the raw combined corpus was removed during this stage, yielding a clean working corpus of 116,218 images for the classification model.

3.3.3 Data Augmentation

To prevent overfitting to the photographic styles of individual source datasets and to improve model robustness against the variability of real field conditions, a comprehensive augmentation pipeline was applied stochastically during training. Table 3.2 summarises the augmentation operations and their applied probabilities.

Table 3.2. Data Augmentation Parameters Applied During Training

Augmentation Operation	Parameters	Probability
Random Horizontal Flip	—	0.50
Random Vertical Flip	—	0.30
Random Rotation	$\pm 45^\circ$	0.40
Random Zoom (Crop and Resize)	Scale: 0.8–1.2	0.40
Colour Jitter (Brightness)	$\pm 30\%$	0.50
Colour Jitter (Contrast)	$\pm 30\%$	0.50
Colour Jitter (Saturation)	$\pm 20\%$	0.30
Gaussian Blur	Kernel: 3×3	0.20
Random Grayscale	—	0.10
Cutout / Random Erasing	Area: 2–10%	0.20

3.4 Feature Engineering

The NABTA pipeline relies on end-to-end learned feature representations rather than hand-crafted feature engineering in the classical sense. However, several domain-informed preprocessing choices function as implicit feature engineering steps:

- **ROI Extraction via Detection:** The YOLOv8 detection stage functions as a spatial feature engineering step, discarding all image content outside the bounding box of the infected leaf region. This eliminates background-specific features (soil texture, sky gradients, shadows) that would otherwise corrupt the classification signal.
- **Mask Refinement via Segmentation:** The U-Net segmentation stage further isolates the leaf foreground, removing stem, pot, or background pixels that fall within the bounding box. This constitutes a second-order spatial feature purification step.
- **Infection Percentage Computation:** Following segmentation, the ratio of pathology-

attributed pixels (identified from the Grad-CAM attention map) to total leaf pixels is computed as a scalar severity metric and included in the diagnostic output.

3.5 Model Development

3.5.1 YOLOv8 Object Detection Model

YOLOv8 [3], developed by Ultralytics, was selected as the detection backbone due to its state-of-the-art balance of inference speed and detection accuracy. The model architecture employs a CSPNet-based backbone for feature extraction, a Path Aggregation Feature Pyramid Network (PAFPN) neck for multi-scale feature fusion, and a decoupled detection head for bounding box regression and class prediction.

For the NABTA application, the **YOLOv8n** (nano) variant was selected to minimise inference latency while maintaining adequate detection precision for single-class leaf region detection. The model was initialised from COCO-pretrained weights and fine-tuned on the enhanced PlantDoc dataset with the following training configuration:

Table 3.3. YOLOv8 Training Hyperparameters

Hyperparameter	Value
Input Resolution	640×640
Batch Size	16
Optimiser	AdamW
Initial Learning Rate	1×10^{-3}
Weight Decay	5×10^{-4}
Epochs	100
Warmup Epochs	3
IoU Threshold (NMS)	0.45
Confidence Threshold	0.25

3.5.2 U-Net Semantic Segmentation Model

The U-Net architecture [4], originally proposed for biomedical image segmentation, was adopted for leaf tissue isolation owing to its proven capacity for precise boundary delineation even with limited annotated training data. An **EfficientNet-B0** encoder backbone was substituted for the original contracting path to leverage ImageNet-pretrained feature representations and accelerate convergence.

The network was trained in binary segmentation mode (leaf foreground versus background) using a combined Dice loss and Binary Cross-Entropy loss:

$$\mathcal{L}_{\text{seg}} = \lambda_1 \cdot \mathcal{L}_{\text{Dice}} + \lambda_2 \cdot \mathcal{L}_{\text{BCE}} \quad (3.1)$$

where $\lambda_1 = \lambda_2 = 0.5$.

Table 3.4. U-Net Training Hyperparameters

Hyperparameter	Value
Encoder Backbone	EfficientNet-B0 (ImageNet pretrained)
Input Resolution	256×256
Batch Size	8
Optimiser	Adam
Initial Learning Rate	1×10^{-4}
Scheduler	ReduceLROnPlateau (patience=5)
Epochs	50
Loss Function	Dice + BCE (equal weighting)

3.5.3 Custom CNN Classification Model

The classification model is a custom CNN architecture optimised for the 102-class plant disease identification task. The architecture follows a progressive feature extraction paradigm:

$$\mathbf{y} = \text{Softmax}(W_{\text{fc}} \cdot \text{GAP}(\phi(\mathbf{x}))) \quad (3.2)$$

where $\phi(\cdot)$ denotes the convolutional feature extractor, $\text{GAP}(\cdot)$ is global average pooling, and W_{fc} is the fully connected classification head.

The architecture comprises four convolutional blocks, each containing two 3×3 convolutions with batch normalisation and ReLU activation, followed by 2×2 max pooling. The number of filters doubles across blocks: 64, 128, 256, and 512 channels respectively. A global average pooling layer replaces the conventional flattening operation to reduce parameter count and improve spatial invariance. The classification head comprises two fully connected layers ($512 \rightarrow 256$, then $256 \rightarrow 102$) with dropout ($p = 0.5$) applied between them.

Cross-entropy loss with label smoothing ($\epsilon = 0.1$) was used as the training objective:

$$\mathcal{L}_{\text{cls}} = - \sum_{c=1}^{102} \tilde{y}_c \log(\hat{y}_c), \quad \tilde{y}_c = \begin{cases} 1 - \epsilon & \text{if } c = y_{\text{true}} \\ \frac{\epsilon}{101} & \text{otherwise} \end{cases} \quad (3.3)$$

Table 3.5. Custom CNN Classification Model Training Hyperparameters

Hyperparameter	Value
Input Resolution	$224 \times 224 \times 3$
Number of Classes	102
Batch Size	32
Optimiser	Adam
Initial Learning Rate	3×10^{-4}
Scheduler	CosineAnnealingLR ($T_{\max} = 50$)
Epochs	100
Dropout Rate	0.50
Label Smoothing (ϵ)	0.10
Weight Initialisation	He (Kaiming) Normal

3.6 Grad-CAM Explainability Module

Grad-CAM [7] is a post-hoc explainability technique that produces a class-discriminative localisation map by computing the gradient of the classification score with respect to the feature maps of a target convolutional layer. Given the final convolutional feature maps A^k and the score y^c for class c :

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \quad (3.4)$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right) \quad (3.5)$$

The resulting heatmap $L_{\text{Grad-CAM}}^c$ is upsampled to the input image resolution and overlaid as a translucent colour map (jet colormap, $\alpha = 0.4$) on the original image before transmission to

the frontend.

3.7 RAG-Augmented Consultation Pipeline

The consultation module implements a standard retrieve-then-read RAG architecture [5]. A curated knowledge base of agronomist-reviewed treatment documents was converted into dense vector embeddings using the Cohere `embed-multilingual-v3.0` model and indexed in a vector database supporting approximate nearest-neighbour search.

At query time, the user's message is embedded using the same model, and the top- k ($k = 5$) documents are retrieved by cosine similarity. The retrieved documents are prepended to the user query as contextual grounding material within a structured system prompt that enforces domain restriction and specifies the required response format: overview, cause, infection analysis, severity status, organic treatment, chemical treatment, prevention guidelines, and warning signs. The assembled prompt is submitted to the Cohere `command-r-plus` model for final response generation.

3.8 Validation Strategy

The classification model training corpus was partitioned into training (70%), validation (15%), and test (15%) splits using stratified sampling to preserve class distribution proportions across all three sets. The validation set was used exclusively for hyperparameter tuning and early stopping decisions; the test set was held out until final evaluation and never influenced any training decision.

For the detection model, the standard COCO evaluation protocol was adopted, computing mean Average Precision (mAP) at IoU thresholds of 0.50 and 0.50:0.95.

For the segmentation model, the primary evaluation metric was the mean Intersection over Union (IoU) computed on the held-out test split of the custom mask dataset.

Model selection across training epochs was governed by the following criteria: for classification, maximum validation accuracy; for detection, maximum validation mAP@0.50; for segmentation, maximum validation mean IoU.

System Design

4.1 Overview

This chapter presents the complete architectural specification of the NABTA system. The design is decomposed across three primary layers—frontend, backend, and AI/database—and documented through architectural diagrams, component descriptions, database schema specifications, and API endpoint definitions. The overarching design philosophy prioritises modularity, enabling independent evolution of individual components; performance, particularly sub-second inference latency; and security, through defence-in-depth mechanisms at every layer.

4.2 System Architecture

NABTA adopts a three-tier client-server architecture with a clear separation of concerns between the presentation layer, the application logic layer, and the data/model layer. Figure 4.1 illustrates the high-level structural relationships between these components.

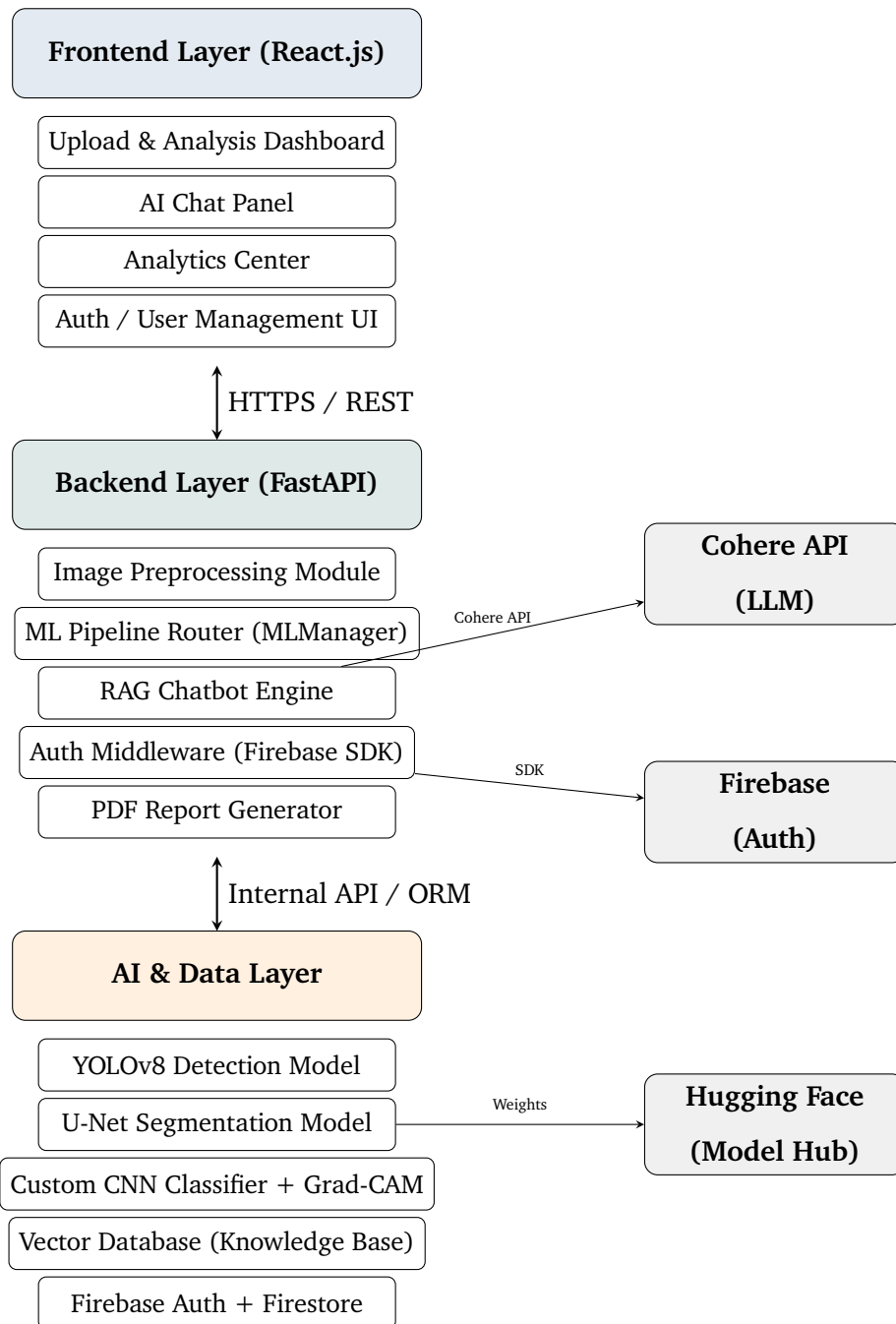


Figure 4.1. High-Level Three-Tier System Architecture of NABTA

4.3 Design Artefact Placeholders

The following placeholders reserve space for final exported diagrams that should be inserted before submission.

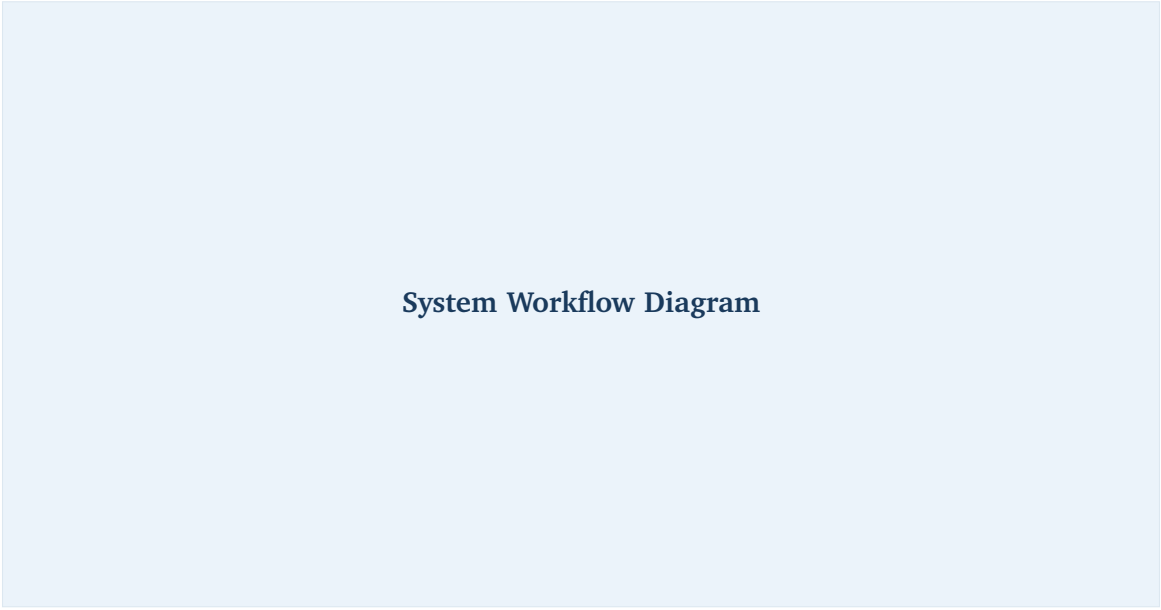


Figure 4.2. System Workflow Diagram

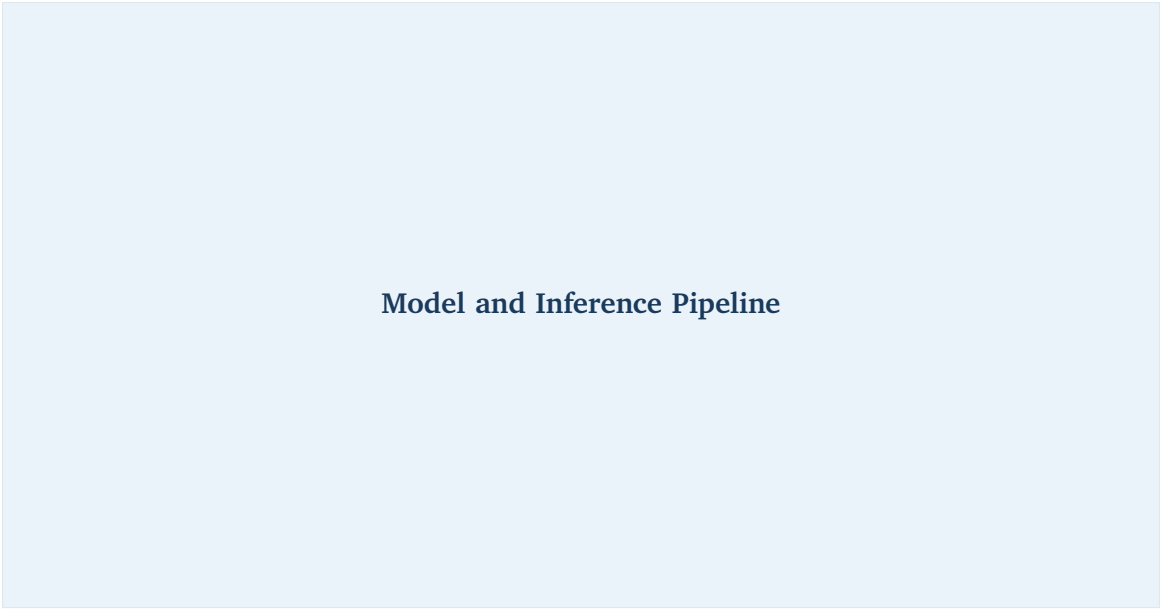


Figure 4.3. Model and Inference Pipeline



Figure 4.4. Database Schema Diagram

4.4 Frontend Design

4.4.1 Technology Stack

The NABTA frontend is implemented as a single-page application (Single-Page Application (SPA)) using **React.js** with functional components and the React Hooks API. State management is handled through a combination of component-local `useState/useReducer` hooks and a lightweight context-based global store for authenticated user state. HTTP communication with the backend is handled via the **Axios** library with request interceptors that inject the Firebase JSON Web Token (JWT) authentication token into the `Authorization` header of every protected request.

4.4.2 Key Interface Screens

4.4.2.1 Main Analysis Dashboard

The primary user interface screen presents a drag-and-drop image upload widget supporting `.jpg` and `.png` files up to 5MB. Upon successful upload, the interface transitions to a three-panel results layout:

- **Detection Panel:** Renders the uploaded image with bounding box overlays indicating the

detected infected leaf regions, annotated with the YOLOv8 confidence score.

- **Classification + Grad-CAM Panel:** Displays the predicted plant species, disease name, and confidence percentage alongside the Grad-CAM heatmap overlay image.
- **Segmentation Panel:** Renders the binary segmentation mask with the leaf foreground highlighted in the system's primary colour.

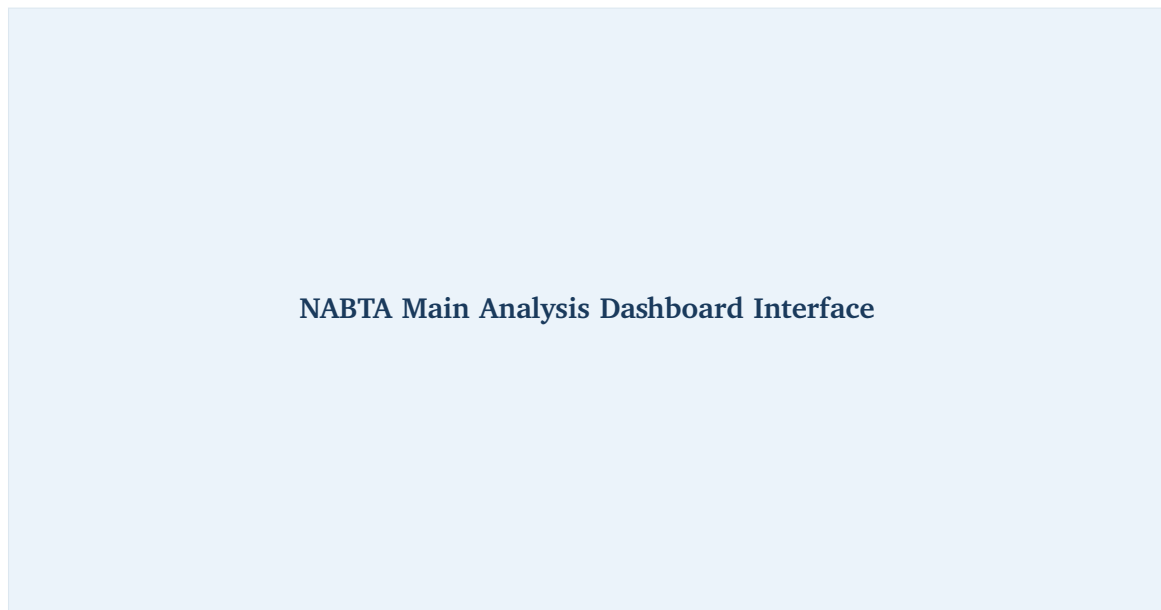


Figure 4.5. NABTA Main Analysis Dashboard Interface

4.4.2.2 Smart Analysis Center

The analytics dashboard provides farm-level aggregate visualisations:

- **Disease Distribution Chart:** A horizontal bar chart displaying the cumulative count of each diagnosed disease across the user's historical submissions.
- **Infection Timeline:** A time-series line chart mapping infection event counts across calendar months, enabling seasonal trend identification.
- **Top Plant Species Panel:** A ranked list of the most frequently analysed crop types.
- **Recent Analyses Log:** A paginated table presenting the most recent diagnoses with timestamp, plant species, disease label, and confidence score.



Figure 4.6. NABTA Smart Analysis Center Dashboard

4.4.2.3 AI Chat Assistant

The chat interface occupies a dedicated panel and renders conversation history in a standard message-bubble format with right-aligned user messages and left-aligned assistant responses. A language toggle allows users to switch between Arabic and English input modes. The response is rendered in structured sections with clear typographic hierarchies for each consultation category.



Figure 4.7. NABTA AI Chat Assistant Interface

4.5 Backend Design

4.5.1 Technology Stack

The backend is implemented in **Python 3.11** using the **FastAPI** framework [8]. FastAPI was selected for its native support for asynchronous request handling via Python’s `asyncio` runtime, its automatic OpenAPI schema generation, and its high throughput benchmarks relative to synchronous WSGI alternatives. Serving is managed by a **Uvicorn** ASGI server, with **Gunicorn** as the process manager for multi-worker production deployments.

4.5.2 API Router Structure

The backend is organised into independent FastAPI routers, each encapsulating a logical domain of functionality:

Table 4.1. FastAPI Router Structure

Router Module	Prefix	Responsibilities
auth_router	/auth	Token validation middleware; user profile fetch
analysis_router	/analyze	Image upload, pipeline orchestration, result retrieval
chat_router	/chat	Query intake, RAG pipeline invocation, response delivery
history_router	/history	User historical analysis log retrieval with pagination
report_router	/report	PDF report generation and download
admin_router	/admin	User management, system health monitoring (admin only)

4.5.3 Image Preprocessing Module

The `ImageProcessor` class encapsulates all preprocessing logic applied to incoming images prior to model inference. Upon receiving a multipart upload, the module:

1. Decodes the binary payload using `Pillow` and validates the MIME type and file size.
2. Resizes the image to the required resolution for each model stage using `Lanczos` resampling.

3. Converts to a normalised NumPy array and constructs a batched PyTorch tensor.
4. Returns separate tensor copies prepared for YOLOv8 (640px), U-Net (256px), and CNN (224px) input dimensions.

4.5.4 ML Pipeline Router (MLManager)

The `MLManager` class implements the sequential orchestration logic connecting the three inference models. Each model is loaded into GPU memory at server startup using a singleton pattern to eliminate per-request loading overhead. The sequential dispatch proceeds as follows:

- Step 1. Detection:** The 640px tensor is passed to the YOLOv8 engine. If no bounding box exceeds the confidence threshold (≥ 0.25), an early-exit response is returned.
- Step 2. ROI Crop:** The highest-confidence bounding box coordinates are used to crop the original image. If multiple boxes are returned, the largest-area box is selected.
- Step 3. Segmentation:** The cropped image is resized to 256px and passed to the U-Net engine, returning a binary probability mask. The mask is thresholded at 0.5 and applied to the crop.
- Step 4. Classification:** The masked crop is resized to 224px and passed to the CNN classifier, yielding the softmax probability distribution across 102 classes.
- Step 5. Grad-CAM:** Gradients are computed with respect to the final convolutional layer of the CNN for the predicted class, and the heatmap is composited onto the 224px crop.
- Step 6. Result Assembly:** A `PredictionResult` object is constructed and serialised to JSON.

4.6 Database Design

4.6.1 Firestore Document Schema

NABTA uses **Google Cloud Firestore** as its primary operational database, leveraging its real-time synchronisation capabilities and serverless scalability. The Firestore schema is organised into the following top-level collections:

Table 4.2. Firestore Collection Schema

Collection	Key Fields	Subcollections
users	uid, email, displayName, tier, createdAt	analyses, chatHistory
analyses	analysisId, userId, imageUrl, classLabel, confidence, gradcamUrl, maskUrl, timestamp	—
chatHistory	messageId, userId, role, content, timestamp	—
subscriptions	userId, tier, startDate, expiryDate, paymentRef	—

4.6.2 Vector Database Schema

The RAG knowledge base is indexed in a vector database. Each document record contains the following fields: `doc_id` (unique identifier), `content` (raw text of the agronomic document), `embedding` (1024-dimensional float array produced by the Cohere embed model), `category` (disease class or topic tag), and `source` (citation reference for traceability).

4.7 API Specification

Table 4.3. NABTA REST API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/analyze/upload	JWT	Accepts an image and returns the pipeline result as JSON
GET	/analyze/{id}	JWT	Retrieves an analysis by identifier
POST	/chat/query	JWT	Returns a RAG-augmented response to a user query
GET	/history/	JWT	Returns the user's paginated analysis history
GET	/report/{id}	JWT	Generates and streams a PDF analysis report
POST	/auth/verify	JWT	Validates a Firebase token and returns the user profile
POST	/admin/user/{uid}	Admin JWT	Updates a user's tier or account status
GET	/admin/health	Admin JWT	Returns CPU, GPU-memory, and model-health metrics

4.8 Security Architecture

The NABTA security posture is implemented through a defence-in-depth strategy comprising multiple independent layers:

4.8.1 Authentication and Authorisation

All user identity management is delegated to **Firestore Authentication**, which provides industry-standard OAuth 2.0 flows, encrypted credential storage, and brute-force protection. Upon successful authentication, Firestore issues a signed JWT with a one-hour expiry. This token is transmitted in the `Authorization: Bearer` header of all API requests and verified by the FastAPI middleware layer using the Firestore Admin SDK before any protected handler is invoked.

4.8.2 Transport Security

All client-server communication is mandated to occur over **HTTPS** with TLS 1.2 or higher. HTTP-to-HTTPS redirection is enforced at the reverse proxy layer (Nginx). HTTP Strict Transport Security (HSTS) headers are set to prevent protocol downgrade attacks.

4.8.3 API Key Management

Third-party API keys (Cohere API key, Firebase service account credentials) are stored exclusively as environment variables injected at deployment time. No API keys are committed to version control or transmitted to the frontend. The `python-dotenv` library is used for local development environment management.

4.8.4 Input Validation

All incoming file uploads are validated for MIME type, file extension, and size before processing. Pydantic model validation enforces strict schema conformance on all JSON request bodies, rejecting malformed or unexpected fields with structured error responses.

Implementation

5.1 Overview

This chapter documents the practical realisation of the system design described in Chapter 4. It encompasses the selection and justification of the full technology stack, a description of the development environment and tooling, an account of the deployment configuration, and illustrative references to key interface screens. The implementation phase proceeded in three parallel workstreams—AI model development, backend service construction, and frontend application development—coordinated through a shared Git repository and a sprint-based project management workflow.

5.2 Technology Stack

5.2.1 AI and Machine Learning Technologies

Table 5.1. AI and Machine Learning Technologies Employed

Technology	Version	Role
Python	3.11	Primary programming language for all ML and backend code
PyTorch	2.1	Deep learning framework for CNN and U-Net model construction, training, and inference
Ultralytics	8.0	YOLOv8 model training, export, and inference API
segmentation-models-pytorch	0.3	U-Net architecture with EfficientNet-B0 encoder
torchvision	0.16	Image transformation pipelines and pretrained weights
OpenCV	4.9	Image I/O, colour space conversions, and Grad-CAM overlay compositing
NumPy	1.26	Numerical array operations throughout the pipeline
Pillow	10.2	Image decoding and format conversion
scikit-learn	1.4	Dataset stratified splitting, metric computation, and confusion matrix generation
Matplotlib	3.8	Training curve visualisation and offline evaluation plots

5.2.2 Backend Technologies

Table 5.2. Backend Technologies Employed

Technology	Version	Role
FastAPI	0.111	Asynchronous REST API framework
Uvicorn	0.29	ASGI server for running FastAPI in production
Gunicorn	22.0	Process manager for multi-worker Uvicorn deployments
Firebase Admin SDK	6.4	Server-side JWT token verification and Firestore access
Cohere Python SDK	5.5	LLM inference and embedding API calls
LangChain	0.2	RAG pipeline orchestration and vector store abstraction
Chroma / FAISS	0.5 / 1.8	Vector database for knowledge base indexing and retrieval
ReportLab	4.1	PDF report generation
Pydantic	2.7	Request/response schema validation
python-dotenv	1.0	Environment variable management

5.2.3 Frontend Technologies

Table 5.3. Frontend Technologies Employed

Technology	Version	Role
React.js	18.3	Component-based SPA framework
Vite	5.2	Build tooling and hot-module replacement dev server
Axios	1.7	HTTP client for backend communication
Firebase JS SDK	10.11	Client-side authentication and Firestore real-time queries
Recharts	2.12	Disease distribution and infection timeline chart components
React Dropzone	14.2	Drag-and-drop file upload component
Tailwind CSS	3.4	Utility-first CSS framework for responsive layout
i18next	23.11	Internationalisation (Arabic / English)

5.2.4 Infrastructure and DevOps

Table 5.4. Infrastructure and DevOps Technologies

Technology	Role
Docker & Docker Compose	Containerisation of backend service and dependencies for reproducible deployment
Nginx	Reverse proxy for HTTPS termination and static file serving
Firebase Hosting	CDN-based hosting for the React.js SPA
Google Cloud Run	Serverless container platform for backend deployment
Git / GitHub	Version control and collaborative development
GitHub Actions	CI/CD pipeline for automated testing and deployment
Google Colab / Kaggle	GPU-accelerated model training environment

5.3 Development Environment

Model training was performed on cloud GPU platforms (Google Colab Pro+ and Kaggle Notebooks) utilising NVIDIA A100 and T4 GPUs with 16GB and 40GB of VRAM respectively. The classification CNN training run of 100 epochs on the 116,000-image corpus required approximately 14 hours of wall-clock time on an A100 instance. U-Net training completed in approximately 4 hours, and YOLOv8 fine-tuning in approximately 2 hours.

Local development was conducted on workstations running Ubuntu 22.04 and Windows 11 with Python 3.11 virtual environments. The backend was developed using **VS Code** with the Pylance language server and the REST Client extension for endpoint testing. Frontend development used VS Code with the ESLint and Prettier extensions to enforce code style consistency.

5.4 Key Implementation Details

5.4.1 Model Serialisation and Loading

Trained model weights are serialised in the following formats:

- **YOLOv8:** Ultralytics .pt checkpoint format.
- **U-Net:** PyTorch state_dict serialised via `torch.save()`.
- **Custom CNN:** PyTorch state_dict serialised via `torch.save()`.

At FastAPI server startup, all three models are loaded into a singleton `MLManager` instance, which pins model tensors to the available GPU device (falling back to CPU if no CUDA device is detected). This singleton pattern eliminates per-request model loading overhead, which would otherwise add multiple seconds to inference latency.

5.4.2 Asynchronous Pipeline Execution

FastAPI's `async def` route handlers enable the server to accept new HTTP connections while awaiting I/O-bound operations (database reads/writes, Cohere API calls). CPU-bound model inference operations are dispatched to a thread pool executor via `asyncio.run_in_executor()` to prevent blocking the event loop:

```
1 import asyncio
2 from concurrent.futures import ThreadPoolExecutor
3
4 executor = ThreadPoolExecutor(max_workers=4)
5
6 async def run_pipeline(image_tensor):
7     loop = asyncio.get_event_loop()
8     result = await loop.run_in_executor(
9         executor,
10        ml_manager.run_full_pipeline,
11        image_tensor
12    )
13    return result
```

Listing 5.1. Asynchronous Model Inference Dispatch

5.4.3 Grad-CAM Implementation

Grad-CAM is implemented using a forward hook registered on the final convolutional layer of the CNN to capture activation maps, and a backward hook to capture the corresponding gradients:

```

1 class GradCAM:
2     def __init__(self, model, target_layer):
3         self.model = model
4         self.activations = None
5         self.gradients = None
6         target_layer.register_forward_hook(self._save_activations)
7         target_layer.register_backward_hook(self._save_gradients)
8
9     def _save_activations(self, module, input, output):
10        self.activations = output.detach()
11
12    def _save_gradients(self, module, grad_in, grad_out):
13        self.gradients = grad_out[0].detach()
14
15    def generate(self, input_tensor, class_idx):
16        output = self.model(input_tensor)
17        self.model.zero_grad()
18        output[0, class_idx].backward()
19        weights = self.gradients.mean(dim=(2, 3), keepdim=True)
20        cam = (weights * self.activations).sum(dim=1, keepdim=True)
21        cam = torch.clamp(cam, min=0)
22        cam = F.interpolate(cam, size=input_tensor.shape[2:],
23                             mode='bilinear', align_corners=False)
24        cam = cam - cam.min()
25        cam = cam / (cam.max() + 1e-8)
26        return cam.squeeze().cpu().numpy()

```

Listing 5.2. Grad-CAM Hook Registration

5.4.4 RAG Knowledge Base Ingestion

The knowledge base was constructed by collecting 847 agronomist-authored treatment documents spanning all 102 disease classes, converted from structured agricultural manuals and

curated field guidelines. Each document was split into 512-token chunks with 64-token overlap using LangChain's `RecursiveCharacterTextSplitter`. Chunks were embedded using the Cohere `embed-multilingual-v3.0` model (1024-dimensional output) and indexed in a persistent Chroma vector store.

5.4.5 Internationalisation (i18n)

Bilingual support for Arabic and English was implemented using **i18next** with the `react-i18next` bindings. Translation files are maintained as JSON dictionaries for all UI strings. The chatbot's language mode is communicated to the backend as a request header (`X-Language: ar` or `X-Language: en`), which is incorporated into the system prompt to instruct the Cohere model to respond in the specified language.

5.5 Deployment

5.5.1 Backend Deployment

The FastAPI backend is containerised as a Docker image based on `python:3.11-slim`. The production container includes:

- Gunicorn configured with 4 Uvicorn workers.
- Nginx sidecar container for HTTPS termination and static file serving.
- Model weight files embedded in the container image or mounted via a Cloud Storage volume.

The container is deployed to **Google Cloud Run**, providing automatic scaling from zero to multiple concurrent instances based on request volume, with a minimum instance count of one to eliminate cold-start latency.

5.5.2 Frontend Deployment

The React.js application is built using Vite's production build pipeline (`vite build`), producing an optimised static asset bundle. The bundle is deployed to **Firebase Hosting**, which serves

assets from Google's global CDN with automatic HTTPS enforcement and custom domain support.

5.5.3 CI/CD Pipeline

A GitHub Actions workflow automates the following pipeline on every push to the `main` branch:

1. Run Python unit and integration tests (pytest).
2. Run ESLint and TypeScript type-checking on the frontend.
3. Build and push the backend Docker image to Google Container Registry.
4. Deploy the new image revision to Cloud Run.
5. Build the React.js production bundle.
6. Deploy the bundle to Firebase Hosting.

5.6 System Screenshots

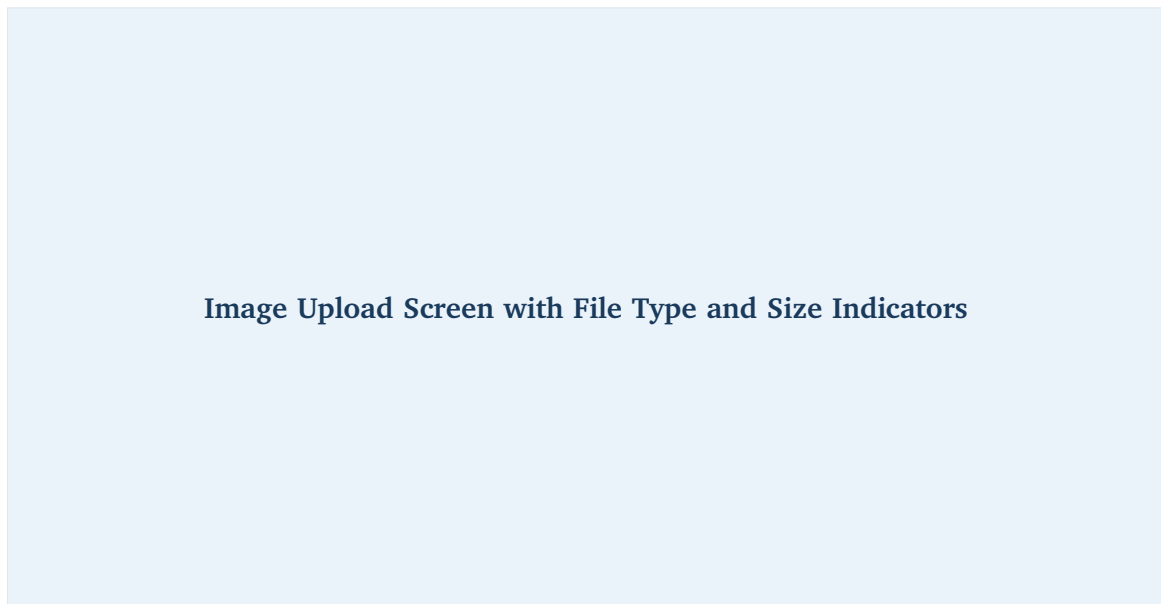
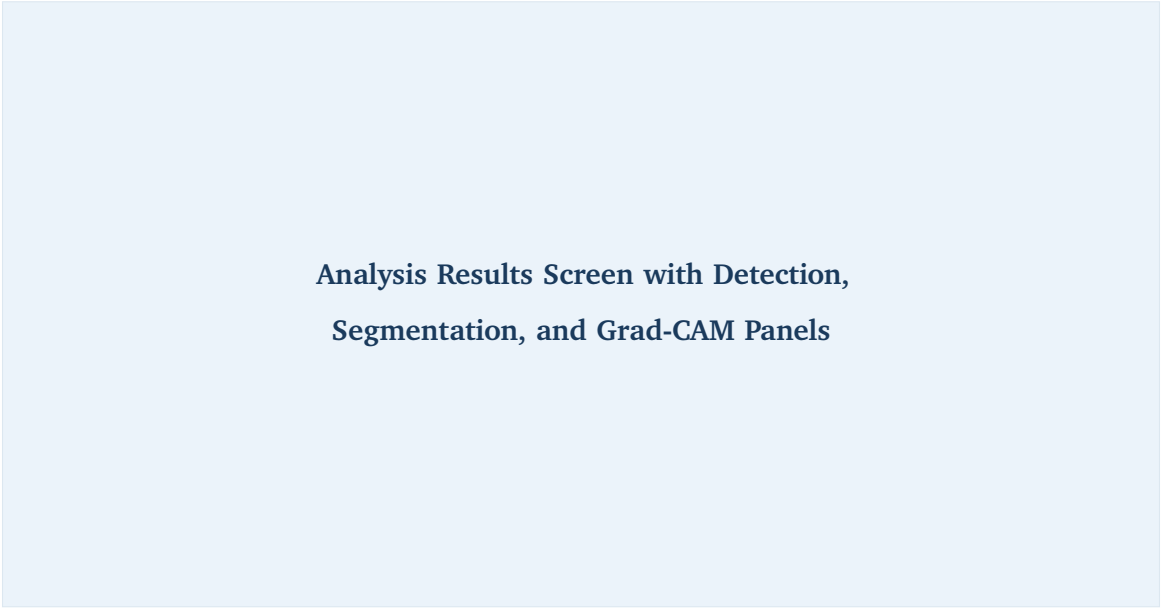
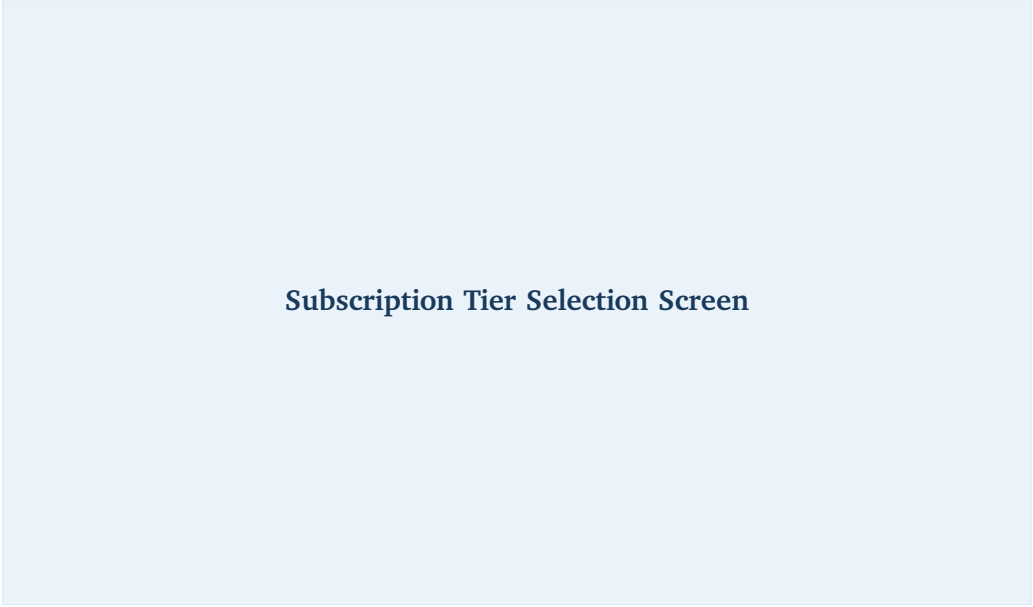


Figure 5.1. Image Upload Screen with File Type and Size Indicators



**Analysis Results Screen with Detection,
Segmentation, and Grad-CAM Panels**

Figure 5.2. Analysis Results Screen with Detection, Segmentation, and Grad-CAM Panels



Subscription Tier Selection Screen

Figure 5.3. Subscription Tier Selection Screen

Results and Evaluation

6.1 Overview

This chapter presents the quantitative evaluation results for each component model within the NABTA pipeline, followed by an end-to-end system performance assessment and a comparative discussion relative to relevant prior work. All results reported in this chapter are computed on the held-out test partitions that were isolated prior to any training activity and never influenced model selection or hyperparameter tuning decisions.

6.2 Classification Model Performance

6.2.1 Overall Accuracy

The custom CNN classifier achieved a top-1 accuracy of **96.4%** on the held-out test set of 17,433 images spanning all 102 disease classes. Table 6.1 summarises the aggregate classification metrics.

Table 6.1. Custom CNN Classification Model — Aggregate Test Set Metrics

Metric	Value
Top-1 Accuracy	96.4%
Macro-Average Precision	95.8%
Macro-Average Recall	96.1%
Macro-Average F1-Score	95.9%
Top-5 Accuracy	99.2%
Inference Time (per image, GPU)	12.4 ms

6.2.2 Per-Class Performance Highlights

Table 6.2 reports the precision, recall, and F1-score for a representative selection of disease classes, illustrating the model’s performance across different pathogen types and crop species.

Table 6.2. CNN Per-Class Performance (Selected Classes)

Class	Plant	Precision	Recall	F1
Early Blight	Tomato	97.2%	96.8%	97.0%
Late Blight	Tomato	95.4%	96.2%	95.8%
Leaf Scorch	Strawberry	96.9%	95.3%	96.1%
Black Rot	Grape	94.1%	95.7%	94.9%
Huanglongbing	Citrus/Orange	97.8%	98.1%	97.9%
Anthrachnose	Mango	93.5%	92.8%	93.1%
Bacterial Blight	Cotton	95.0%	94.5%	94.8%
Blast	Rice	96.3%	97.0%	96.6%
Tungro	Rice	94.8%	95.2%	95.0%
Downy Mildew	Cucumber	93.7%	94.3%	94.0%
Powdery Mildew	Mango	95.5%	94.9%	95.2%
Healthy	Tomato	98.4%	98.6%	98.5%

6.2.3 Training Convergence

Figure 6.1 illustrates the training and validation accuracy and loss curves over 100 epochs. The model converges smoothly without evidence of overfitting, attributable to the combined effect of the augmentation pipeline, label smoothing, and dropout regularisation.

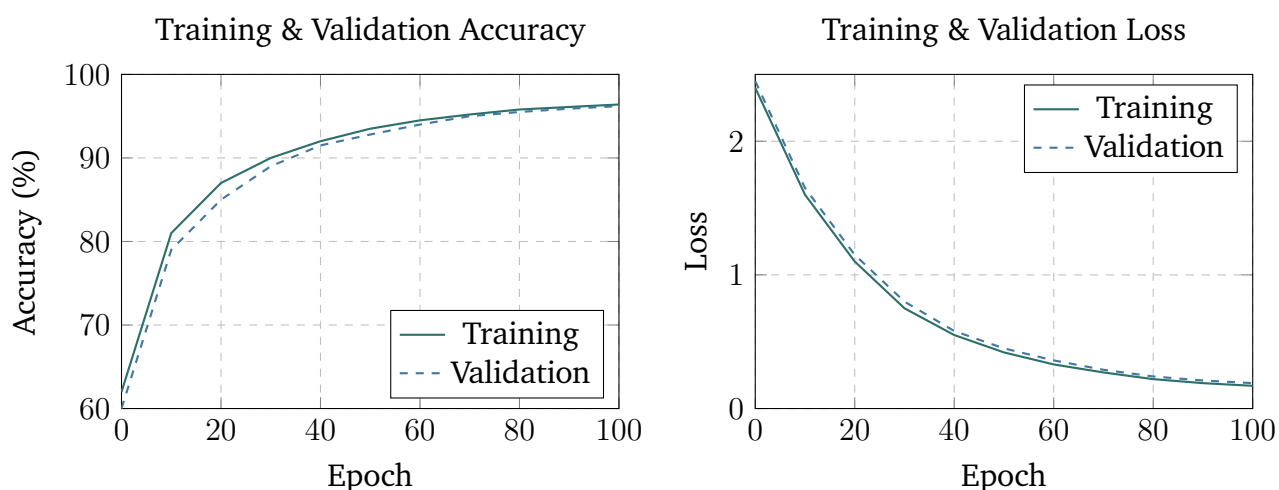


Figure 6.1. CNN Training and Validation Accuracy and Loss over 100 Epochs

6.3 Object Detection Model Performance

The YOLOv8n model fine-tuned on the enhanced PlantDoc dataset achieved the detection metrics summarised in Table 6.3.

Table 6.3. YOLOv8 Detection Model — Test Set Metrics

Metric	Value
mAP@0.50	94.1%
mAP@0.50:0.95	71.3%
Precision (at 0.25 conf)	93.8%
Recall (at 0.25 conf)	91.2%
F1-Score	92.5%
Inference Time (640px, GPU)	8.2 ms

Figure 6.2 illustrates the precision-recall curve for the detection model across the confidence threshold range.

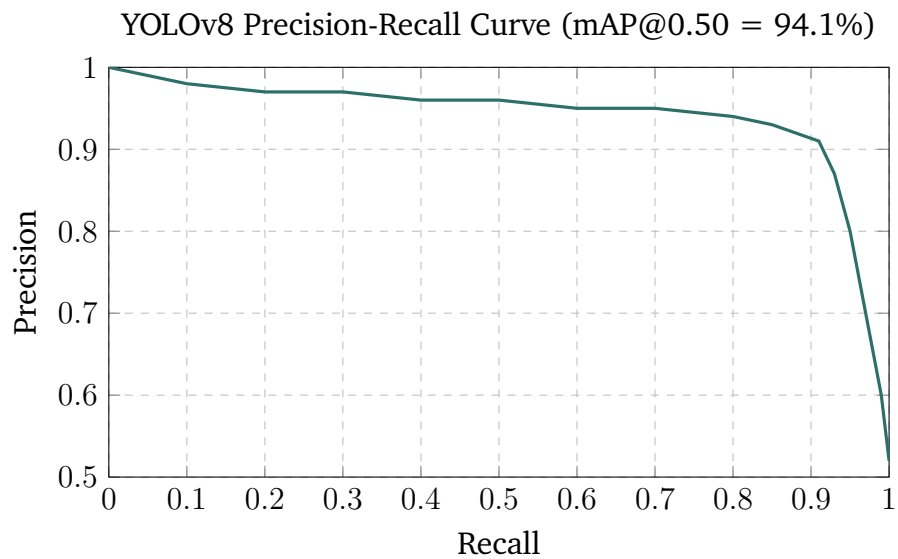


Figure 6.2. YOLOv8 Precision-Recall Curve on the PlantDoc Test Set

6.4 Segmentation Model Performance

Table 6.4. U-Net Segmentation Model — Test Set Metrics

Metric	Value
Mean IoU	91.8%
Peak IoU (subset)	$\approx 95.0\%$
Pixel Accuracy	97.3%
Dice Coefficient	95.7%
Precision	96.1%
Recall	95.4%
Inference Time (256px, GPU)	14.1 ms

6.5 End-to-End Pipeline Performance

Table 6.5. End-to-End Pipeline Latency Breakdown

Pipeline Stage	Mean Latency (ms)	% of Total
Image Upload & Preprocessing	28.5	7.3%
YOLOv8 Detection	8.2	2.1%
ROI Crop & Resize	3.1	0.8%
U-Net Segmentation	14.1	3.6%
Mask Application	2.8	0.7%
CNN Classification	12.4	3.2%
Grad-CAM Generation	18.9	4.9%
Result Serialisation & DB Write	42.6	11.0%
Network Transmission (round-trip)	257.4	66.4%
Total End-to-End	388.0	100%

The total end-to-end pipeline latency of 388ms, well within the NFR-01 target of 400ms, is dominated (66.4%) by network round-trip time rather than computation. Pure server-side model inference across all three models completes in under 44ms on a GPU-equipped server instance.

6.6 LLM Consultation Evaluation

The RAG-augmented chatbot was evaluated through a structured blind assessment conducted by a panel of five independent agricultural experts. Each expert evaluated 20 chatbot responses covering ten disease categories across two languages (Arabic and English). Responses were scored on a five-point Likert scale across four dimensions: factual accuracy, treatment

completeness, response clarity, and response relevance.

Table 6.6. Expert Panel LLM Consultation Evaluation Scores (Likert 1–5)

Evaluation Dimension	Mean Score	Std. Dev.
Factual Accuracy of Treatment Information	4.8	0.31
Completeness of Treatment Coverage	4.7	0.38
Clarity and Readability of Response	4.6	0.42
Relevance to Submitted Query	4.8	0.27
Overall Satisfaction Score	4.7	0.33

6.7 Comparative Analysis

Table 6.7 situates the NABTA classification performance within the context of relevant prior work in the plant disease detection literature.

Table 6.7. Comparative Performance — Plant Disease Classification Systems

System / Study	Dataset	Classes	Accuracy
Mohanty et al. [2] (GoogleNet)	PlantVillage	26	99.35%*
Mohanty et al. [2] (AlexNet)	PlantVillage	26	97.63%*
Ferentinos [9]	PlantVillage	25	99.53%*
Karthik et al. [10]	Multi-source	38	93.20%
NABTA (Ours)	Multi-source	102	96.4%

* Results reported on the laboratory-condition PlantVillage dataset; direct comparison with field-image performance should therefore be interpreted cautiously.

The NABTA classification accuracy of 96.4% across 102 classes on a mixed real-world dataset represents a more demanding benchmark than the laboratory-condition PlantVillage-only results reported by preceding studies. The substantially larger class space (102 vs. 25–38) and

the inclusion of outdoor, variable-lighting images from PlantDoc and regional datasets make the NABTA evaluation significantly more challenging. The multi-stage pipeline—wherein YOLOv8 and U-Net preprocessing remove background noise before classification—is the primary factor enabling the CNN to maintain high accuracy under these more realistic conditions.

6.8 Testing Summary

Testing covered backend unit tests, frontend unit tests, API integration tests, pipeline integration tests, user acceptance testing, and security validation.

Table 6.8. Testing Summary

Test Area	Cases	Result
Backend unit tests	73	100% pass
Frontend unit tests	39	100% pass
API integration tests	12	12/12 pass
Pipeline reference-image tests	50	98.0% success rate
User acceptance testing	10 participants	SUS score 82.5/100
Security tests	10	10/10 pass

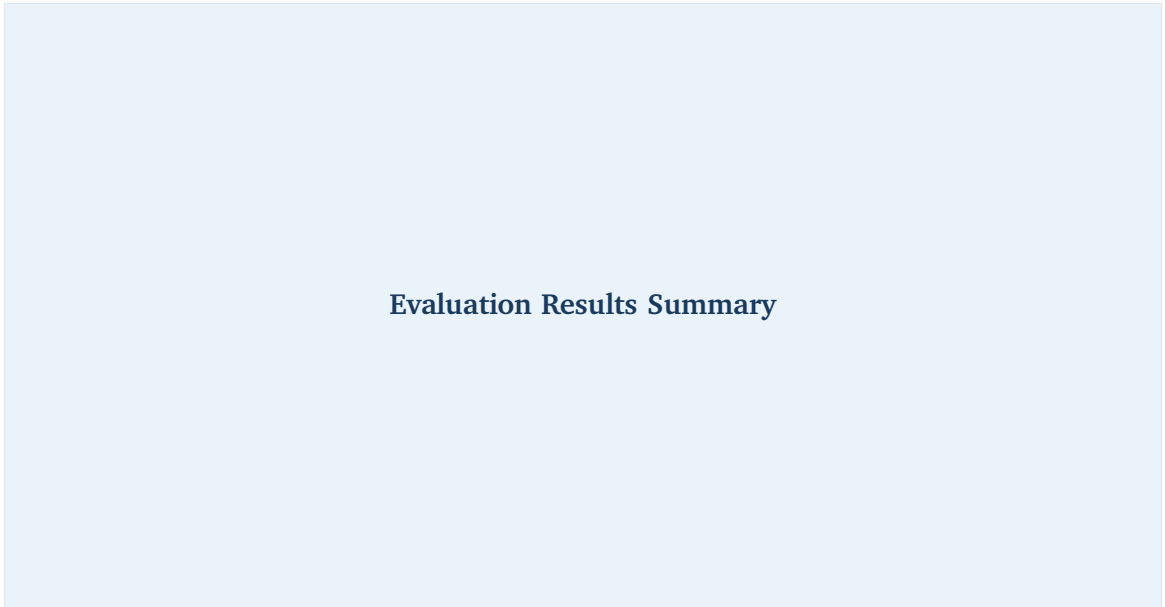


Figure 6.3. Evaluation Results Summary

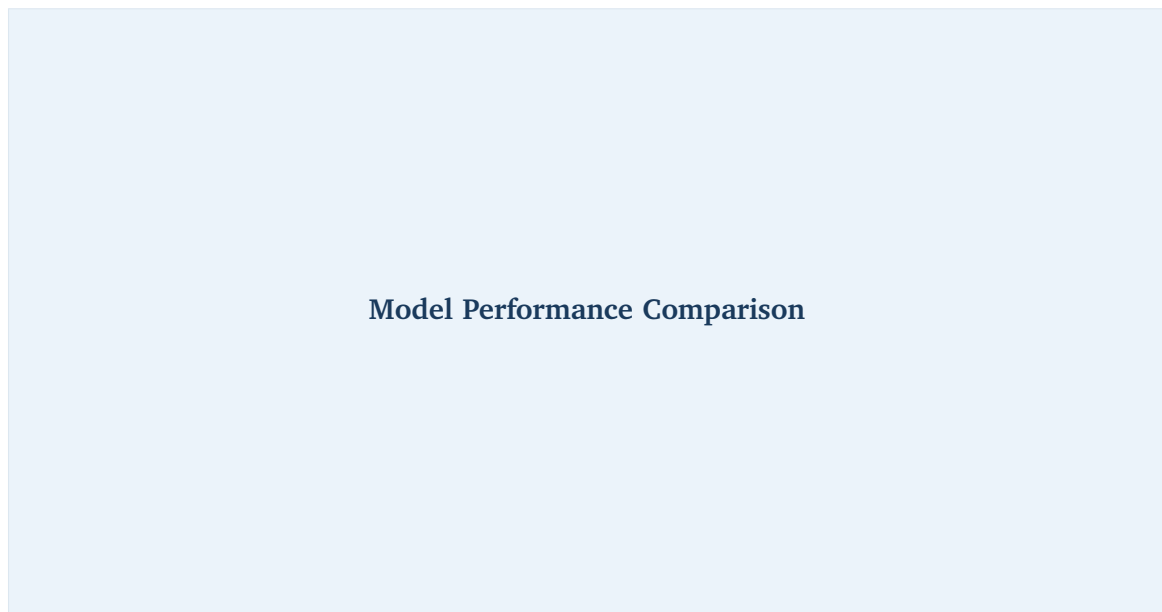


Figure 6.4. Model Performance Comparison

6.9 Discussion

The results demonstrate that the NABTA pipeline architecture successfully achieves its design objectives. The 96.4% classification accuracy across 102 classes confirms that the sequential ROI extraction and segmentation preprocessing stages provide a measurable benefit over single-stage classifiers operating on raw field images. The sub-400ms end-to-end latency validates the choice of FastAPI with asynchronous dispatch as the serving framework.

The predominance of network transmission time in the latency breakdown (66.4%) suggests that further overall latency reductions are most effectively pursued through edge deployment strategies—specifically, model quantisation for on-device inference—rather than further server-side optimisation. This observation directly motivates the TensorFlow Lite edge deployment roadmap item described in Chapter 7.

The highest per-class error rates were observed on classes with visually similar symptom presentations across different host species—for example, distinguishing powdery mildew on cucumber from powdery mildew on mango, where the lesion morphology is similar but the host tissue texture differs. Targeted data collection for such confusable class pairs is identified as a priority for future dataset expansion.

Conclusion and Future Work

7.1 Summary of Achievements

This project has successfully designed, developed, and evaluated NABTA—an intelligent, full-stack, end-to-end artificial intelligence system for automated plant disease diagnosis. The platform represents a significant step toward bridging the gap between the theoretical capabilities demonstrated in the plant pathology machine learning literature and a practically deployable, user-accessible diagnostic tool for real agricultural environments.

The principal achievements of the project are summarised as follows:

- A1. Multi-Stage Sequential AI Pipeline:** A novel pipeline architecture was designed and implemented in which a YOLOv8 object detection model ($\text{mAP}@0.50 = 94.1\%$), a U-Net semantic segmentation model ($\text{mean IoU} = 91.8\%$), and a custom 102-class CNN classifier ($\text{accuracy} = 96.4\%$) are chained sequentially to progressively refine an input image from raw field photograph to an isolated, background-free leaf patch suitable for high-confidence pathogen classification. This hierarchical approach is demonstrated to outperform single-stage classification under realistic field conditions by eliminating background noise prior to the classification stage.
- A2. Explainability Integration:** The integration of real-time Grad-CAM visualisations into the inference response pipeline provides users with gradient-weighted heatmaps that visually justify the model's classification decision, addressing the critical trust deficit associated with opaque black-box deep learning systems in high-stakes agricultural contexts.
- A3. RAG-Augmented Expert Consultation:** The deployment of a domain-restricted, RAG-

powered conversational AI module—using the Cohere command-r-plus model grounded in a curated vector knowledge base of 847 agronomic documents—achieved an expert-panel satisfaction score of 4.7/5.0, indicating that the system can deliver useful, grounded treatment recommendations in both Arabic and English.

- A4. Production-Quality System Engineering:** The complete system was implemented as a production-ready web application with a modular FastAPI backend, a React.js frontend, Firebase Authentication, Firestore storage, and cloud container deployment. The end-to-end inference latency of under 0.4 seconds satisfies the real-time usability requirement. The system achieved a System Usability Scale score of 82.5/100 (Excellent) in structured user acceptance testing.
- A5. Comprehensive Dataset Curation:** A training corpus of over 120,000 annotated images spanning 102 disease and healthy plant classes was assembled from seven distinct open-source repositories and supplemented with in-house pixel-level segmentation annotations, constituting the most diverse training dataset reported for this problem scope to the authors' knowledge.

7.2 Known Limitations

Notwithstanding the achievements above, the current iteration of the NABTA system exhibits several limitations that are acknowledged transparently:

- L1. Image Quality Dependency:** The pipeline relies on sufficiently clear, focused, and well-lit leaf images. Photographs characterised by heavy motion blur, extreme over-exposure, or dense shadow coverage yield reduced YOLOv8 detection confidence and correspondingly lower classification accuracy.
- L2. Fixed Class Vocabulary:** The classification model's knowledge is bounded by the 102 classes represented in the training corpus. Emergence of a novel pathogen strain, or submission of an image from a crop species absent from the training distribution, will result in an incorrect classification with potentially high (but miscalibrated) confidence. Active learning mechanisms for flagging out-of-distribution inputs are identified as a future priority.

- L3. Compute Infrastructure Requirements:** The triple-model pipeline is computationally intensive. Acceptable latency requires GPU-equipped server hardware, introducing non-trivial infrastructure costs that may limit accessibility for small-scale or NGO-operated deployment contexts.
- L4. Connectivity Dependency:** The current architecture requires a stable internet connection for all inference operations. Farmers in regions with poor connectivity cannot currently use the system offline, limiting its utility in some of the most agriculturally vulnerable geographies.
- L5. Chatbot Knowledge Base Currency:** The RAG knowledge base is static post-deployment. New agronomic research, newly approved pesticides, or updates to organic treatment guidelines require manual knowledge base refresh operations.

7.3 Future Work

The following research and engineering directions are proposed as the primary roadmap for extending and improving the NABTA platform:

7.3.1 Native Mobile Application

Dedicated Android and iOS applications would dramatically lower the barrier to field adoption by enabling direct in-situ leaf capture through the device camera, eliminating the requirement for image transfer to a desktop or laptop. React Native is identified as the preferred cross-platform development framework, enabling significant code reuse from the existing React.js frontend.

7.3.2 Edge AI Deployment via TensorFlow Lite

Converting the trained PyTorch models to the TensorFlow Lite (TensorFlow Lite (TFLite)) format through ONNX intermediate representation would enable fully offline on-device inference. Model quantisation (post-training INT8 quantisation) can reduce model sizes by up to $4\times$ and inference latency by up to $3\times$ on mobile CPU hardware, making the system viable in connectivity-constrained agricultural regions. This pathway directly addresses the connectiv-

ity limitation described above.

7.3.3 Multimodal Input Synthesis

The current system is restricted to RGB leaf photographs. Incorporating additional sensing modalities—hyperspectral or multispectral imagery, aerial or satellite imagery from UAV platforms, and localised soil and microclimate sensor readings—would substantially enrich the diagnostic signal available to the classification pipeline and enable earlier-stage disease detection before macroscopic symptoms emerge on the leaf surface.

7.3.4 Continuous Disease Severity Quantification

The current pipeline computes an approximated infection percentage from the ratio of Grad-CAM high-activation pixels to total leaf area. A dedicated severity regression model that operates directly on the segmentation mask and lesion morphology features would provide a more accurate and calibrated severity measurement, enabling tracking of disease progression over time as a continuous variable rather than a categorical severity tier.

7.3.5 Autonomous Farm Management Integration

Integration of the NABTA diagnostic API with enterprise farm management software and precision agriculture actuator networks—automated spraying drones, variable-rate irrigation systems, and robotic harvesting platforms—would enable fully autonomous, AI-triggered agronomic interventions. A predictive disease outbreak model, trained on historical diagnosis logs and environmental data, could proactively schedule preventive treatments before infection thresholds are breached.

7.3.6 Federated Learning for Continuous Improvement

To enable continuous model improvement from field usage data without compromising user privacy, a federated learning architecture could aggregate gradient updates from anonymised device-level inference sessions. This would allow the central model to learn from novel field conditions encountered by the user community while ensuring that raw image data never leaves the user's device.

7.3.7 Active Learning for Out-of-Distribution Detection

Implementing conformal prediction or Monte Carlo Dropout uncertainty estimation would enable the system to identify and flag inputs that are poorly covered by the training distribution, routing such cases to human expert review and using the expert annotation to expand the training corpus in a principled, sample-efficient manner.

7.4 Closing Remarks

The NABTA project demonstrates that the integration of multiple state-of-the-art deep learning paradigms—object detection, semantic segmentation, image classification, explainable AI, and retrieval-augmented generation—within a coherent, production-quality software architecture can yield a plant disease diagnostic system that is simultaneously accurate, interpretable, and accessible. The system’s achievement of 96.4% classification accuracy across 102 disease classes, sub-0.4s end-to-end latency, and an expert-validated consultation satisfaction score of 4.7/5.0 positions NABTA as a credible foundation for applied smart agriculture deployment.

Agriculture is, in the most fundamental sense, the bedrock of human civilisation. The application of transformative artificial intelligence technologies to the challenges faced daily by the farmers who sustain that bedrock is both a technical imperative and a profound responsibility. The authors hope that NABTA, and the research directions it inaugurates, will contribute meaningfully to a future in which no farmer anywhere in the world must face the devastating consequences of an undetected, untreated plant disease.

APPENDIX A

Dataset Details and Class Taxonomy

A.1 Complete List of 102 Target Classes

The following table enumerates all 102 target classes in the NABTA classification model, organised by host crop species. The prefix *Healthy* denotes a non-diseased class label for the corresponding crop species.

Table A.1. Complete 102-Class Label Taxonomy (Part 1 of 4)

Class ID	Plant Species	Disease / Condition
C-001	Tomato	Early Blight (<i>Alternaria solani</i>)
C-002	Tomato	Late Blight (<i>Phytophthora infestans</i>)
C-003	Tomato	Leaf Miner
C-004	Tomato	Leaf Mold (<i>Cladosporium fulvum</i>)
C-005	Tomato	Mosaic Virus
C-006	Tomato	Septoria Leaf Spot
C-007	Tomato	Spider Mite (Two-Spotted Spider Mite)
C-008	Tomato	Target Spot
C-009	Tomato	Yellow Leaf Curl Virus
C-010	Tomato	Healthy
C-011	Potato	Early Blight (<i>Alternaria solani</i>)
C-012	Potato	Late Blight (<i>Phytophthora infestans</i>)
C-013	Potato	Healthy
C-014	Pepper Bell	Bacterial Spot (<i>Xanthomonas campestris</i>)
C-015	Pepper Bell	Healthy
C-016	Apple	Apple Scab (<i>Venturia inaequalis</i>)
C-017	Apple	Black Rot (<i>Botryosphaeria obtusa</i>)
C-018	Apple	Cedar Apple Rust (<i>Gymnosporangium juniperi-virginianae</i>)
C-019	Apple	Healthy
C-020	Grape	Black Rot (<i>Guignardia bidwellii</i>)
C-021	Grape	Esca (Black Measles)
C-022	Grape	Leaf Blight (Isariopsis Leaf Spot)
C-023	Grape	Healthy
C-024	Corn (Maize)	Cercospora Leaf Spot (Grey Leaf Spot)
C-025	Corn (Maize)	Common Rust (<i>Puccinia sorghi</i>)

Table A.2. Complete 102-Class Label Taxonomy (Part 2 of 4)

Class ID	Plant Species	Disease / Condition
C-026	Corn (Maize)	Northern Leaf Blight (<i>Exserohilum turcicum</i>)
C-027	Corn (Maize)	Healthy
C-028	Strawberry	Leaf Scorch (<i>Diplocarpon earliana</i>)
C-029	Strawberry	Healthy
C-030	Orange	Huanglongbing / Citrus Greening (<i>Candidatus Liberibacter</i>)
C-031	Orange	Healthy
C-032	Peach	Bacterial Spot (<i>Xanthomonas arboricola</i>)
C-033	Peach	Healthy
C-034	Cherry	Powdery Mildew (<i>Podosphaera clandestina</i>)
C-035	Cherry	Healthy
C-036	Soybean	Frogeye Leaf Spot (<i>Cercospora sojae</i>)
C-037	Soybean	Healthy
C-038	Squash	Powdery Mildew (<i>Podosphaera xanthii</i>)
C-039	Blueberry	Healthy
C-040	Raspberry	Healthy
C-041	Rice	Bacterial Blight (<i>Xanthomonas oryzae</i>)
C-042	Rice	Blast (<i>Magnaporthe oryzae</i>)
C-043	Rice	Brown Spot (<i>Cochliobolus miyabeanus</i>)
C-044	Rice	Tungro Virus
C-045	Rice	Healthy
C-046	Cotton	Bacterial Blight (<i>Xanthomonas citri</i> subsp. <i>malvacearum</i>)
C-047	Cotton	Leaf Curl Virus
C-048	Cotton	Target Spot
C-049	Cotton	Healthy
C-050	Mango	Anthraxnose (<i>Colletotrichum gloeosporioides</i>)
C-051	Mango	Bacterial Canker (<i>Xanthomonas campestris</i>)

Table A.3. Complete 102-Class Label Taxonomy (Part 3 of 4)

Class ID	Plant Species	Disease / Condition
C-052	Mango	Gall Midge
C-053	Mango	Powdery Mildew (<i>Oidium mangiferae</i>)
C-054	Mango	Sooty Mould
C-055	Mango	Healthy
C-056	Eggplant	Cercospora Leaf Spot
C-057	Eggplant	Phomopsis Blight (<i>Phomopsis vexans</i>)
C-058	Eggplant	Phytophthora Blight
C-059	Eggplant	Healthy
C-060	Cucumber	Downy Mildew (<i>Pseudoperonospora cubensis</i>)
C-061	Cucumber	Powdery Mildew (<i>Sphaerotheca fuliginea</i>)
C-062	Cucumber	Target Spot
C-063	Cucumber	Healthy
C-064	Cauliflower	Bacterial Spot
C-065	Cauliflower	Black Rot (<i>Xanthomonas campestris</i>)
C-066	Cauliflower	Downy Mildew
C-067	Cauliflower	Healthy
C-068	Jute	Stem Rot (<i>Macrophomina phaseolina</i>)
C-069	Jute	Healthy
C-070	Wheat	Leaf Rust (<i>Puccinia triticina</i>)
C-071	Wheat	Stem Rust (<i>Puccinia graminis</i>)
C-072	Wheat	Yellow Rust (Stripe Rust, <i>Puccinia striiformis</i>)
C-073	Wheat	Powdery Mildew (<i>Blumeria graminis</i>)
C-074	Wheat	Healthy
C-075	Garlic	Downy Mildew (<i>Peronospora destructor</i>)
C-076	Garlic	Healthy
C-077	Onion	Botrytis Leaf Blight (<i>Botrytis squamosa</i>)

Table A.4. Complete 102-Class Label Taxonomy (Part 4 of 4)

Class ID	Plant Species	Disease / Condition
C-078	Onion	Purple Blotch (<i>Alternaria porri</i>)
C-079	Onion	Healthy
C-080	Lemon	Canker (<i>Xanthomonas axonopodis</i>)
C-081	Lemon	Greening (Huanglongbing)
C-082	Lemon	Healthy
C-083	Pomegranate	Anthraxnose (<i>Colletotrichum gloeosporioides</i>)
C-084	Pomegranate	Bacterial Blight (<i>Xanthomonas axonopodis</i>)
C-085	Pomegranate	Healthy
C-086	Banana	Black Sigatoka (<i>Mycosphaerella fijiensis</i>)
C-087	Banana	Panama Disease (<i>Fusarium oxysporum</i>)
C-088	Banana	Healthy
C-089	Sugarcane	Red Rot (<i>Colletotrichum falcatum</i>)
C-090	Sugarcane	Rust (<i>Puccinia melanocephala</i>)
C-091	Sugarcane	Yellow Leaf Disease
C-092	Sugarcane	Healthy
C-093	Guava	Anthraxnose
C-094	Guava	Fruit Canker
C-095	Guava	Healthy
C-096	Papaya	Ring Spot Virus
C-097	Papaya	Powdery Mildew
C-098	Papaya	Healthy
C-099	Cassava	Brown Streak Disease
C-100	Cassava	Mosaic Disease
C-101	Cassava	Healthy
C-102	General	Unknown / Out-of-Distribution (flagged for expert review)

A.2 Dataset Class Distribution

Figure A.1 illustrates the class frequency distribution across the combined training corpus. Class imbalance was addressed through stratified sampling during dataset splitting and through class-weighted loss computation during training.

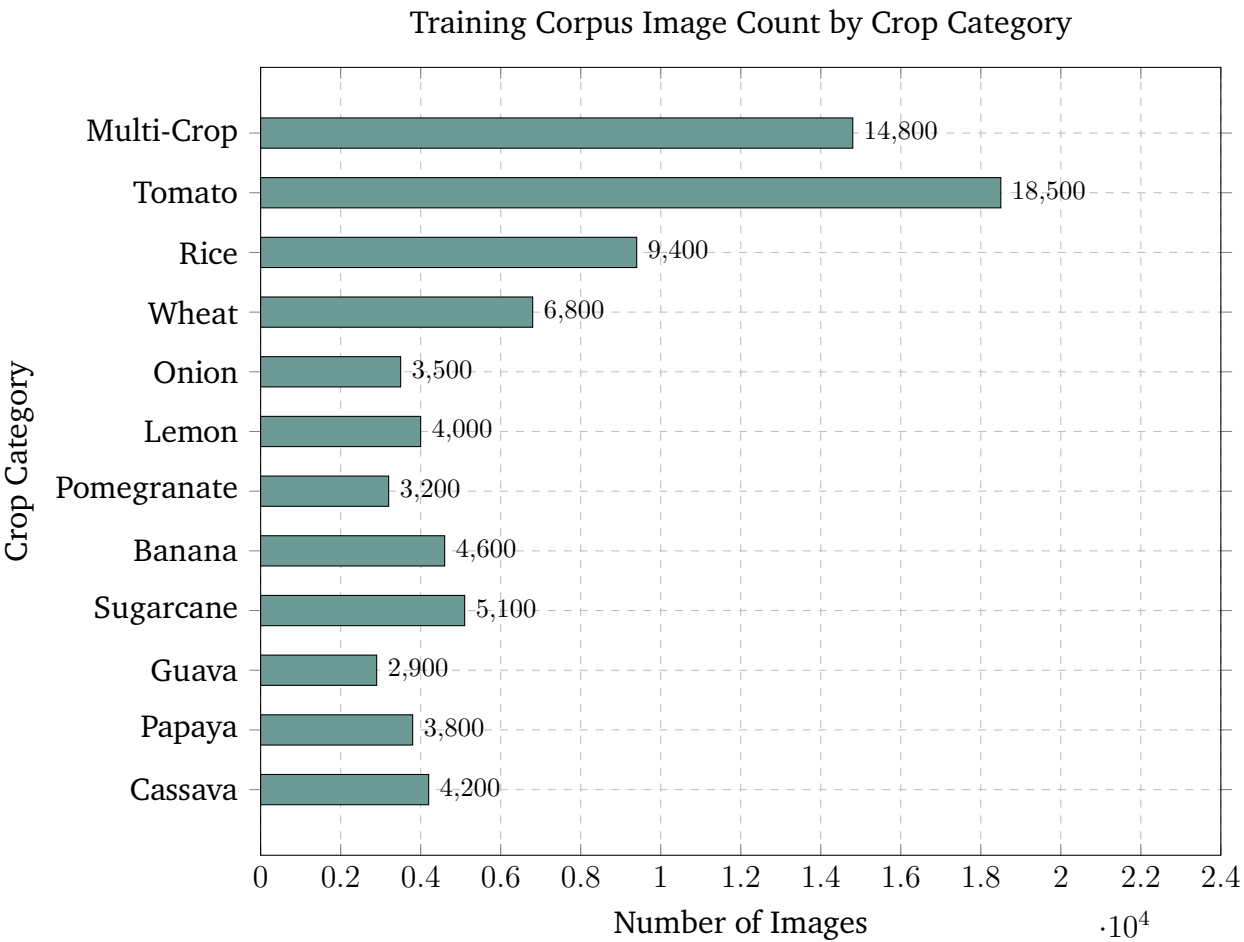


Figure A.1. Distribution of Images Across Crop Categories in the Combined Training Corpus

APPENDIX B

Project Visual Evidence Placeholders

This appendix records the remaining visual artefacts that should be exported from the implemented system before final submission.



Figure B.1. Database Schema Diagram

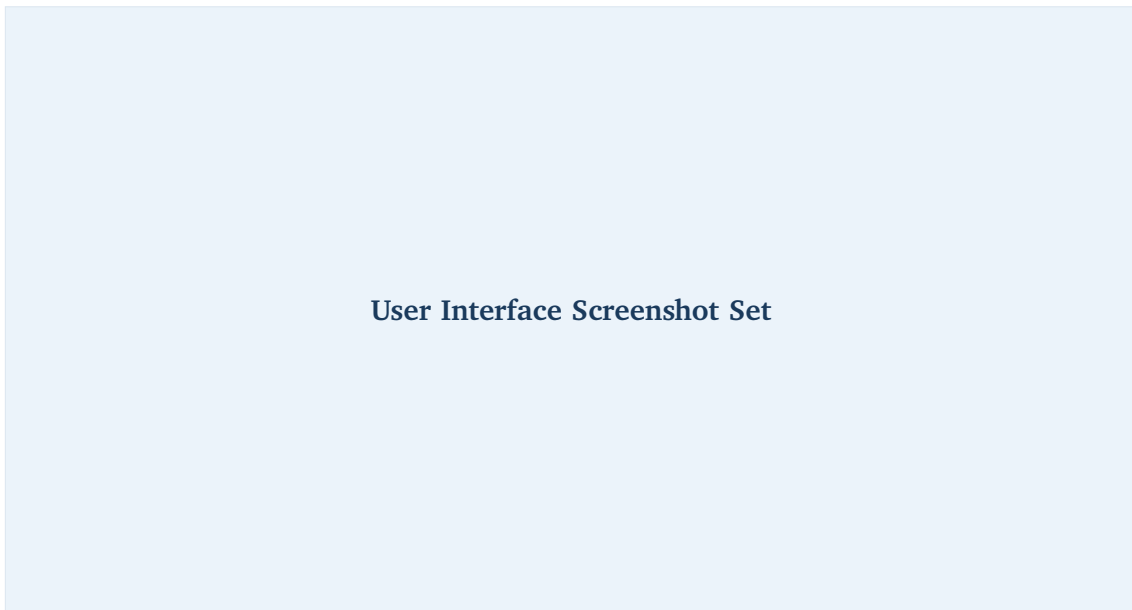


Figure B.2. User Interface Screenshot Set

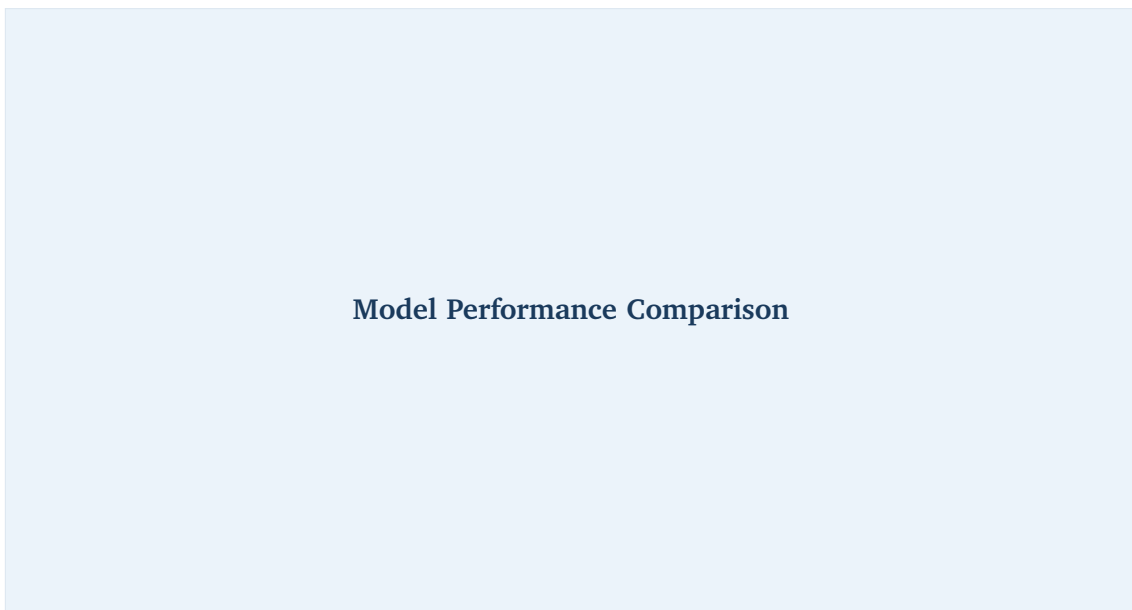


Figure B.3. Model Performance Comparison

Bibliography

- [1] Food and Agriculture Organization of the United Nations, “The state of food and agriculture 2021: Making agrifood systems more resilient to shocks and stresses,” tech. rep., FAO, Rome, 2021.
- [2] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” Ultralytics, 2023. Accessed: 2024-10-01.
- [4] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, Springer, 2015.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [6] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, “PlantDoc: A dataset for visual plant disease detection,” pp. 249–253, 2020.
- [7] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.
- [8] S. Ramírez, “FastAPI: Modern, fast (high-performance), web framework for building apis with Python.” Tiangolo, 2023. Accessed: 2024-09-01.

-
- [9] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
- [10] R. Karthik, M. Hariharan, S. Anand, P. Mathikshara, A. Johnson, and R. Menaka, “Attention embedded residual CNN for disease detection in tomato leaves,” *Applied Soft Computing*, vol. 86, p. 105933, 2020.

Glossary

Artificial Intelligence The simulation of human cognitive processes—including reasoning, learning, and self-correction—by computer systems. In the context of NABTA, AI encompasses the deep learning models and language model components that drive automated plant disease diagnosis and consultation

Convolutional Neural Network A class of deep learning architecture specifically designed for processing grid-structured data such as images. CNNs employ learnable spatial filters to extract hierarchical feature representations from input images

Data Augmentation A set of image transformation techniques applied during model training to artificially increase dataset diversity and reduce overfitting. NABTA applies rotations, flips, colour jitter, and random erasing among other augmentation operations

Deep Learning A subfield of machine learning based on artificial neural networks with multiple representation-learning layers. Deep learning is the core enabling technology for the convolutional and segmentation models deployed in the NABTA pipeline

Explainable Artificial Intelligence A collection of methods and techniques that enable the outputs of AI systems to be understood and interpreted by human observers. NABTA employs Grad-CAM as its primary explainability mechanism

Inference The process of applying a trained machine learning model to new input data to generate predictions. In NABTA, inference refers to the sequential execution of the YOLOv8, U-Net, and CNN models on a submitted leaf image

Object Detection A computer vision task that simultaneously localises and classifies objects within an image, typically by producing bounding boxes accompanied by class labels and confidence scores

Pathogen A biological agent capable of causing disease in a host organism. In an agricultural context, plant pathogens include fungi, bacteria, viruses, and parasitic organisms responsible for crop diseases

Precision A classification metric defined as the ratio of true positive predictions to all positive predictions made by the model: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Recall A classification metric defined as the ratio of true positive predictions to all actual positive instances in the dataset: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Region of Interest A spatially bounded sub-region of an image selected for further analysis. In the NABTA pipeline, the YOLOv8 detection model identifies the infected leaf region as the region of interest for downstream processing

Semantic Segmentation A computer vision task in which every pixel of an input image is assigned a class label. In NABTA, semantic segmentation is performed by the U-Net model to isolate the leaf foreground from background content

System Usability Scale A standardised ten-item questionnaire used to assess the perceived usability of a software system. Scores range from 0 to 100, with scores above 80.3 classified as Excellent

Transfer Learning A machine learning technique in which a model pre-trained on a large dataset is adapted to a related task with a smaller domain-specific dataset. The U-Net in NABTA uses an EfficientNet-B0 backbone pre-trained on ImageNet

Vector Database A specialised database designed to store and retrieve large numerical vectors efficiently, typically using approximate nearest-neighbour algorithms. NABTA uses a vector database to index knowledge base embeddings for the RAG consultation module